??: miniprogram/app.js

```
const USER_INFO_KEY = 'userInfo'
const LOGOUT_FLAG_KEY = 'manualLoggedOut'
const RESET_KEYS = [
  USER_INFO_KEY,
  LOGOUT_FLAG_KEY,
  'selectedRegion',
  'userLocation',
  'recentRegionList',
]

function getSettingAsync() {
  return new Promise((resolve, reject) => {
    wx.getSetting({
      success: resolve,
      fail: reject,
    })
  })
}

function getLocationAsync(timeout = 8000) {
  return new Promise((resolve, reject) => {
    const timer = setTimeout(() => {
      reject(new Error('getLocation timeout'))
    }, timeout)

    wx.getLocation({
      type: 'gcj02',
      success: (res) => {
        clearTimeout(timer)
        resolve(res)
      },
      fail: (err) => {
        clearTimeout(timer)
        reject(err)
      },
    })
  })
}

App({
  globalData: {
    userInfo: null,
    templateId: {},
  },

  onLaunch(options) {
    wx.cloud.init({
      env: 'cloud1-3ghmr5ki7b1172fe',
      traceUser: true,
    })

    this.bootstrap(options)
  },

  onShow() {
    this.refreshUserLocationCache()
  },

  async bootstrap(options = {}) {
    try {
```

```
      const targetUrl = this.getLaunchTarget()

      if (targetUrl) {
        setTimeout(() => {
          wx.reLaunch({ url: targetUrl })
        }, 80)
      }
    } catch (err) {
      console.error('bootstrap failed', err)
    }
  },

  async refreshUserLocationCache() {
    try {
      const setting = await getSettingAsync()
      if (!setting || !setting.authSetting || !setting.authSetting['scope.userLocation'] ) {
        return
      }

      const locationRes = await getLocationAsync()
      const userLocation = {
        latitude: locationRes.latitude,
        longitude: locationRes.longitude,
      }

      wx.setStorageSync('userLocation', userLocation)

      const currentUserInfo = this.globalData.userInfo || wx.getStorageSync(USER_INFO_KEY)
      if (currentUserInfo && currentUserInfo.openid) {
        const nextUserInfo = {
          ...currentUserInfo,
          userLocation,
          locationAuthorized: true,
        }
        this.globalData.userInfo = nextUserInfo
        wx.setStorageSync(USER_INFO_KEY, nextUserInfo)
      }
    } catch (err) {
      console.warn('refreshUserLocationCache failed', err)
    }
  },

  getLaunchTarget() {
    const agreePrivacy = wx.getStorageSync('agreePrivacy')
    if (!agreePrivacy) {
      return '/pages/welcome/welcome'
    }

    if (wx.getStorageSync(LOGOUT_FLAG_KEY)) {
      this.globalData.userInfo = null
      return '/pages/home/home'
    }

    const userInfo = this.getUserInfo()
    if (!userInfo) {
      return '/pages/home/home'
    }

    this.globalData.userInfo = userInfo
    return this.getPostLoginTarget(userInfo)
```

```
  },

  getPostLoginTarget(userInfo) {
    if (userInfo.onboardingCompleted) {
      return '/pages/home/home'
    }

    if (!userInfo.profileCompleted) {
      return '/pages/registerProfile/registerProfile?mode=register'
    }

    if (!userInfo.locationChoiceMade) {
      return '/pages/permissionSettings/permissionSettings?mode=onboarding'
    }

    if (!userInfo.dnaCompleted) {
      return '/pages/dnaTag/dnaTag?mode=setup'
    }

    return '/pages/home/home'
  },

  setUserInfo(userInfo) {
    if (!userInfo || !userInfo.openid) {
      this.clearUserInfo({ markLoggedOut: false })
      return
    }

    const currentUserInfo = this.globalData.userInfo || wx.getStorageSync(USER_INFO_KEY) || {}
    const mergedUserInfo = {
      ...currentUserInfo,
      ...userInfo,
    }

    if (!userInfo.avatarPreviewUrl && currentUserInfo.avatarPreviewUrl && currentUserInfo.avatarUrl === mergedUserInfo.avatarUrl) {
      mergedUserInfo.avatarPreviewUrl = currentUserInfo.avatarPreviewUrl
    }

    if (!userInfo.avatarResolvedUrl && currentUserInfo.avatarResolvedUrl && currentUserInfo.avatarUrl === mergedUserInfo.avatarUrl) {
      mergedUserInfo.avatarResolvedUrl = currentUserInfo.avatarResolvedUrl
    }

    this.globalData.userInfo = mergedUserInfo
    wx.setStorageSync(USER_INFO_KEY, mergedUserInfo)
    wx.removeStorageSync(LOGOUT_FLAG_KEY)
  },

  clearUserInfo(options = {}) {
    const { markLoggedOut = true } = options
    this.globalData.userInfo = null
    wx.removeStorageSync(USER_INFO_KEY)
    if (markLoggedOut) {
      wx.setStorageSync(LOGOUT_FLAG_KEY, true)
    } else {
      wx.removeStorageSync(LOGOUT_FLAG_KEY)
    }
  },

  getUserInfo() {
```

```
    if (wx.getStorageSync(LOGOUT_FLAG_KEY)) {
      return null
    }

    const userInfo = this.globalData.userInfo || wx.getStorageSync(USER_INFO_KEY)
    if (!userInfo || !userInfo.openid) {
      return null
    }

    return userInfo
  },

  hasActiveSession(options = {}) {
    const userInfo = this.getUserInfo()
    if (!userInfo) {
      return false
    }

    return true
  },

  clearLocalTestData() {
    this.globalData.userInfo = null
    RESET_KEYS.forEach((key) => {
      wx.removeStorageSync(key)
    })
  },
})
```

??: miniprogram/app.json

```
{
  "pages": [
    "pages/home/home",
    "pages/message/message",
    "pages/cart/cart",
    "pages/me/me",
    "pages/welcome/welcome",
    "pages/login/login",
    "pages/notePublish/notePublish",
    "pages/noteDetail/noteDetail",
    "pages/messageInteraction/messageInteraction",
    "pages/messageMerchant/messageMerchant",
    "pages/messageMerchantChat/messageMerchantChat",
    "pages/messagePlatform/messagePlatform",
    "pages/messagePlatformSupport/messagePlatformSupport",
    "pages/messagePlatformGuarantee/messagePlatformGuarantee",
    "pages/messageBuddyApply/messageBuddyApply",
    "pages/messageBuddyApplyChat/messageBuddyApplyChat",
    "pages/messageConversation/messageConversation",
    "pages/permissionSettings/permissionSettings",
    "pages/registerProfile/registerProfile",
    "pages/dnaTag/dnaTag",
    "pages/buddyIntent/buddyIntent",
    "pages/activityDetail/activityDetail",
    "pages/productDetail/productDetail",
    "pages/scenicDetail/scenicDetail",
    "pages/hotelDetail/hotelDetail",
    "pages/search/search",
    "pages/searchResults/searchResults",
    "pages/askXiaohe/askXiaohe",
    "pages/askXiaoheChat/askXiaoheChat",
    "pages/regionSelect/regionSelect",
    "pages/book/book",
    "pages/order/order",
    "pages/travelerSelect/travelerSelect",
    "pages/travelerEdit/travelerEdit",
    "pages/myActivities/myActivities",
    "pages/activityOrderDetail/activityOrderDetail",
    "pages/activityReview/activityReview",
    "pages/myOrders/myOrders",
    "pages/reviewCenter/reviewCenter",
    "pages/favorites/favorites",
    "pages/favoritesEdit/favoritesEdit",
    "pages/ruralFun/ruralFun",
    "pages/academy/academy",
    "pages/knowledgeDetail/knowledgeDetail",
    "pages/hotel/hotel",
    "pages/mall/mall",
    "pages/qaCenter/qaCenter"
  ],
  "permission": {
    "scope.userLocation": {
      "desc": "\u7528\u4e8e\u4e3a\u60a8\u63a8\u8350\u9644\u8fd1\u9152\u5e97\u6c11\u5bbf\u3001\u6d3b\u52a8\u4e0e\u666f\u70b9\uff0c\u5e76\u663e\u793a\u5f53\u524d\u6240\u5728\u57ce\u5e02"
    }
  },
  "requiredPrivateInfos": [
    "getLocation",
    "chooseLocation"
```

```
  ],
  "window": {
    "navigationBarTitleText": "\u519c\u65c5e\u7ad9",
    "navigationBarBackgroundColor": "#F8F8F8",
    "navigationBarTextStyle": "black"
  },
  "lazyCodeLoading": "requiredComponents",
  "tabBar": {
    "color": "#999999",
    "selectedColor": "#57984a",
    "list": [
      { "pagePath": "pages/home/home", "text": "\u9996\u9875" },
      { "pagePath": "pages/message/message", "text": "\u6d88\u606f" },
      { "pagePath": "pages/cart/cart", "text": "\u8d2d\u7269\u8f66" },
      { "pagePath": "pages/me/me", "text": "\u6211\u7684" }
    ]
  }
}
```

??: miniprogram/app.wxss

```
/**app.wxss**/
:root {
  --theme-page-bg: #f6faf4;
  --theme-panel-bg: #ffffff;
  --theme-soft-bg: #edf5ea;
  --theme-soft-bg-strong: #e4f0df;
  --theme-primary: #57984a;
  --theme-primary-dark: #27692a;
  --theme-primary-light: #dff0dc;
  --theme-text-main: #1f2a1f;
  --theme-text-sub: #667566;
  --theme-text-muted: #99a598;
  --theme-border: #dbe8d7;
  --theme-danger: #d9544f;
}

.container {
  display: flex;
  flex-direction: column;
  align-items: center;
  box-sizing: border-box;
}

button {
  background: initial;
}

button:focus{
  outline: 0;
}

button::after{
  border: none;
}


page {
  background: var(--theme-page-bg);
  color: var(--theme-text-main);
  display: flex;
  flex-direction: column;
  justify-content: flex-start;
}
```

??: miniprogram/envList.js

```
const envList = [];
const isMac = false;
module.exports = {
  envList,
  isMac
};
```

??: miniprogram/sitemap.json

```
{
  "desc": "关于本文件的更多信息，请参考文档 https://developers.weixin.qq.com/miniprogram/dev/framework/sitemap.html",
  "rules": [{
    "action": "allow",
    "page": "*"
  }]
}
```

??: miniprogram/config/agent.js

```
module.exports = {
  yuxiaoheBotId: "agent-yuxiaohe-1grmumm967563411",
  yuxiaoheEnvId: "cloud1-3ghmr5ki7b1172fe",
  yuxiaoheServiceName: "agent-yuxiaohe-1grmumm967563411",
  yuxiaoheHttpBaseUrl: "https://cloud1-3ghmr5ki7b1172fe-1403917845.ap-shanghai.app.tcloudbase.com"
}
```

??: miniprogram/components/cloudTipModal/index.js

```
Component({

  /**
   * 页面的初始数据
   */
  data: {
    showTip: false,
  },
  properties: {
    showTipProps: Boolean,
    title:String,
    content:String
  },
  observers: {
    showTipProps: function(showTipProps) {
      this.setData({
        showTip: showTipProps
      });
    }
  },
  methods: {
    onClose(){
      this.setData({
        showTip: !this.data.showTip
      });
    },
  }
});
```

??: miniprogram/components/cloudTipModal/index.json

```
1 {
2   "usingComponents": {},
3   "component": true
4 }
```

??: miniprogram/components/cloudTipModal/index.wxml

```
<!--miniprogram/components/cloudTipModal/index.wxml-->
<!--  wx:if="{{showUploadTip}}" -->
<view class="install_tip" wx:if="{{showTip}}">
  <view class="install_tip_back"></view>
  <view class="install_tip_detail">
    <image class="install_tip_close" bind:tap="onClose" src="../../images/icons/close.png"/>
    <view class="install_tip_detail_title">{{title}}</view>
    <view class="install_tip_detail_tip">{{content}}</view>
  </view>
</view>
```

??: miniprogram/components/cloudTipModal/index.wxss

```
.install_tip_back {
  position: fixed;
  top: 0;
  right: 0;
  bottom: 0;
  left: 0;
  background-color: rgba(0,0,0,0.4);
  z-index: 1;
}
.install_tip_close{
  position:absolute;
  right: 10rpx;
  top: 10rpx;
  width: 32px;
  height: 32px;
  /* background-color: red; */
}
.install_tip_detail {
  position: fixed;
  background-color: white;
  right: 0;
  bottom: 0;
  left: 0;
  border-radius: 40rpx 40rpx 0 0;
  padding: 50rpx 50rpx 100rpx 50rpx;
  z-index: 9;
}

.install_tip_detail_title {
  font-weight: 400;
  font-size: 40rpx;
  text-align: center;
}

.install_tip_detail_tip {
  font-size: 25rpx;
  color: rgba(0,0,0,0.4);
  margin-top: 20rpx;
  text-align: left;
}
.install_tip_detail_buttons {
  padding-top: 50rpx;
  display: flex;
}
.install_tip_detail_button {
  color: #07C160;
  font-weight: 500;
  background-color: rgba(0,0,0,0.1);
  width: 40%;
  text-align: center;
  /* height: 90rpx; */
  /* line-height: 90rpx; */
  border-radius: 10rpx;
  margin: 0 auto;
}

.install_tip_detail_button_primary {
  background-color: #07C160;
  color: #fff;
}
```

??: miniprogram/utils/activityCoverTags.js

```
const GENERIC_TAGS = new Set([
  '\u4e61\u6751',
  '\u519c\u65c5',
  '\u6587\u65c5',
  '\u4f53\u9a8c',
  '\u6d3b\u52a8',
  '\u7f8e\u98df',
  '\u73a9\u6cd5',
  '\u65b9\u5f0f',
  '\u51fa\u884c',
  '\u51fa\u6e38',
  '\u5fae\u5ea6\u5047',
  '\u5468\u672b\u77ed\u9014',
  '\u6444\u5f71',
  '\u89c2\u5149',
  '\u624b\u4f5c',
  '\u975e\u9057',
  '\u91c7\u6458',
  '\u82b1\u6d77',
  '\u7530\u56ed',
  '\u7267\u573a',
  '\u6c11\u5bbf',
  '\u9152\u5e97',
  '\u5065\u5eb7\u517b\u751f',
  '\u4eb2\u5b50\u7814\u5b66',
  '\u51fa\u7247\u6253\u5361',
])

const NOISE_WORDS = [
  '\u4f53\u9a8c',
  '\u4e4b\u65c5',
  '\u4e4b\u884c',
  '\u4e3b\u9898',
  '\u6587\u5316',
  '\u98ce\u60c5',
  '\u9650\u5b9a',
  '\u6d3b\u52a8',
  '\u7ebf\u8def',
  '\u5fae\u5ea6\u5047',
  '\u6162\u6e38',
  '\u89c2\u5149',
  '\u6253\u5361',
  '\u73a9\u6cd5',
  '\u65b9\u5f0f',
  '\u9002\u5408',
  '\u53ef\u9009',
  '\u4f18\u5148',
  '\u53ef\u62fc\u8f66',
  '\u62fc\u8f66',
  '\u8f7b\u677e',
]

const NONE_STAY_PATTERNS = [
  '\u4e0d\u542b\u4f4f\u5bbf',
  '\u65e0\u4f4f\u5bbf',
  '\u4e0d\u542b\u4f4f',
  '\u65e0\u4f4f',
]

function normalizeText(value = '') {
```

```
  return String(value || '').trim()
}

function normalizeList(value) {
  return Array.isArray(value) ? value.map(normalizeText).filter(Boolean) : []
}

function containsAny(text, patterns) {
  return patterns.some((pattern) => text.includes(pattern))
}

function normalizeCompareText(value) {
  let text = normalizeText(value)
  text = text.replace(/\u534a\u65e5/gu, '\u534a\u5929')
  text = text.replace(/\u4e00\u65e5/gu, '1\u5929')
  text = text.replace(/\u519c\u5bb6\u5348\u9910/gu, '\u519c\u5bb6\u98ce\u5473\u9910')
  text = text.replace(/\u519c\u5bb6\u665a\u9910/gu, '\u519c\u5bb6\u98ce\u5473\u9910')
  text = text.replace(/\u519c\u5bb6\u996d/gu, '\u519c\u5bb6\u98ce\u5473\u9910')
  text = text.replace(/\u4e61\u6751\u5348\u9910/gu, '\u519c\u5bb6\u98ce\u5473\u9910')
  text = text.replace(/\u5de5\u574a/gu, '\u624b\u4f5c')
  text = text.replace(/(\u624b\u4f5c)+/gu, '\u624b\u4f5c')
  text = text.replace(/[\u00b7\u30fb\u3001\uff0c,\s\/|｜-]+/gu, '')
  NOISE_WORDS.forEach((word) => {
    text = text.split(word).join('')
  })
  return text
}

function isSimilarTag(candidate, existingList) {
  const normalizedCandidate = normalizeCompareText(candidate)
  if (!normalizedCandidate) {
    return false
  }

  return existingList.some((item) => {
    const normalizedItem = normalizeCompareText(item)
    if (!normalizedItem) {
      return false
    }

    return (
      normalizedItem === normalizedCandidate ||
      normalizedItem.includes(normalizedCandidate) ||
      normalizedCandidate.includes(normalizedItem)
    )
  })
}

function pushCandidate(target, candidate, excludedList = [], max = 5) {
  const text = normalizeText(candidate)
  if (!text || target.length >= max || GENERIC_TAGS.has(text)) {
    return
  }

  if (isSimilarTag(text, excludedList) || isSimilarTag(text, target)) {
    return
  }

  target.push(text)
}

function appendCandidates(target, candidates, excludedList, max, limit) {
```

```
  const local = []
  candidates.forEach((candidate) => {
    pushCandidate(local, candidate, excludedList, limit)
  })
  local.forEach((candidate) => {
    pushCandidate(target, candidate, excludedList, max)
  })
}

function buildDurationTag(days, durationText) {
  if (typeof days === 'number' && days >= 2) {
    return `${days}\u5929${days - 1}\u665a`
  }

  if (typeof days === 'number' && days > 0 && days < 1) {
    return normalizeText(durationText) || '\u534a\u5929'
  }

  if (typeof days === 'number' && days > 0) {
    return `${days}\u5929`
  }

  return normalizeText(durationText) || '1\u5929'
}

function collectServiceTags(activity, target, excludedList, max) {
  const transportText = normalizeText(activity.transport)
  const pickupPoint = normalizeText(activity.pickupPoint)
  const candidates = []

  if (transportText) {
    if (transportText.includes('\u4fdd\u59c6\u8f66')) {
      candidates.push('\u4fdd\u59c6\u8f66')
    }
    if (transportText.includes('\u4e00\u5355\u4e00\u56e2')) {
      candidates.push('\u4e00\u5355\u4e00\u56e2')
    }
    if (containsAny(transportText, ['\u4e13\u8f66', '\u5305\u8f66'])) {
      candidates.push('\u4e13\u8f66\u670d\u52a1')
    }
    if (containsAny(transportText, ['\u5546\u52a1\u8f66'])) {
      candidates.push('\u5546\u52a1\u8f66\u63a5\u9001')
    }
    if (containsAny(transportText, ['\u5c0f\u5df4', '\u5927\u5df4', '\u5df4\u58eb', '\u65c5\u6e38\u5df4\u58eb', '\u56e2\u961f\u5df4\u58eb'])) {
      candidates.push('\u5df4\u58eb\u63a5\u9001')
    }
    if (containsAny(transportText, ['\u81ea\u9a7e'])) {
      candidates.push('\u81ea\u9a7e\u53ef\u9009')
    }
  }

  if (activity.pickup) {
    if (containsAny(pickupPoint, ['\u9ad8\u94c1', '\u706b\u8f66\u7ad9', '\u8f66\u7ad9', '\u7ad9'])) {
      candidates.push('\u7ad9\u70b9\u63a5\u9001')
    } else if (containsAny(pickupPoint, ['\u9152\u5e97', '\u6c11\u5bbf'])) {
      candidates.push('\u4f4f\u5bbf\u63a5\u9001')
    } else if (containsAny(pickupPoint, ['\u5546\u5708', '\u5730\u94c1', '\u57ce\u5173', '\u65b0\u533a'])) {
      candidates.push('\u5546\u5708\u63a5\u9001')
    } else {
```

```
      candidates.push('\u5b9a\u70b9\u63a5\u9001')
    }
  }

  appendCandidates(target, candidates, excludedList, max, 2)
}

function collectStayTags(activity, target, excludedList, max) {
  const stayText = normalizeText(activity.stay)
  if (!stayText || containsAny(stayText, NONE_STAY_PATTERNS)) {
    return
  }
  const candidates = []

  if (containsAny(stayText, ['\u7cbe\u54c1\u6c11\u5bbf', '\u7279\u8272\u6c11\u5bbf'])) {
    candidates.push('\u7279\u8272\u6c11\u5bbf')
  } else if (stayText.includes('\u6c11\u5bbf')) {
    candidates.push('\u6c11\u5bbf\u53ef\u9009')
  }

  if (containsAny(stayText, ['\u9152\u5e97', '\u53cc\u5e8a', '\u5927\u5e8a'])) {
    candidates.push('\u9152\u5e97\u53ef\u9009')
  }

  if (containsAny(stayText, ['\u5e10\u7bf7', '\u9732\u8425'])) {
    candidates.push('\u5e10\u7bf7\u9732\u8425')
  }

  if (containsAny(stayText, ['\u6728\u5c4b', '\u5c0f\u9662'])) {
    candidates.push('\u4e61\u91ce\u6728\u5c4b')
  }

  appendCandidates(target, candidates, excludedList, max, 1)
}

function collectFoodTags(textPool, target, excludedList, max) {
  const joined = textPool.join(' ')
  const candidates = []

  if (joined.includes('\u6cb3\u5dde') && containsAny(joined, ['\u4e61\u5473\u5bb4', '\u98ce\u5473\u5bb4'])) {
    candidates.push('\u6cb3\u5dde\u4e61\u5473\u5bb4')
  }

  if (joined.includes('\u767e\u5408') && containsAny(joined, ['\u5168\u5e2d', '\u5bb4'])) {
    candidates.push('\u767e\u5408\u5168\u5e2d\u5bb4')
  } else if (joined.includes('\u767e\u5408') && containsAny(joined, ['\u7f8e\u98df', '\u98ce\u5473', '\u751c\u54c1', '\u624b\u4f5c'])) {
    candidates.push('\u767e\u5408\u98ce\u5473')
  }

  if (joined.includes('\u73ab\u7470') && containsAny(joined, ['\u5bb4', '\u665a\u9910', '\u5348\u9910', '\u98ce\u5473'])) {
    candidates.push('\u73ab\u7470\u98ce\u5473\u5bb4')
  }

  if (containsAny(joined, ['\u519c\u5bb6\u5348\u9910', '\u519c\u5bb6\u996d', '\u519c\u5bb6\u9910', '\u4e61\u6751\u665a\u9910'])) {
    candidates.push('\u519c\u5bb6\u98ce\u5473\u9910')
  }

```

```
  if (containsAny(joined, ['\u8017\u725b\u9178\u5976', '\u9178\u5976'])) {
    candidates.push('\u8017\u725b\u9178\u5976')
  }

  if (containsAny(joined, ['\u85cf\u5f0f\u5976\u8336', '\u5976\u8336'])) {
    candidates.push('\u85cf\u5f0f\u5976\u8336')
  }

  appendCandidates(target, candidates, excludedList, max, 1)
}

function normalizeHighlightTag(value) {
  let text = normalizeText(value)
  if (!text) {
    return ''
  }

  if (text.includes('\u767e\u5408') && text.includes('\u5c0f\u5361')) {
    return '\u767e\u5408\u5c0f\u5361'
  }

  text = text.replace(/^[0-9]+\.\s*/u, '')
  text = text
    .replace(/^\u8d60\u9001/u, '')
    .replace(/^\u9644\u8d60/u, '')
    .replace(/^\u5b9a\u5236/u, '')
    .replace(/\u4f53\u9a8c$/u, '')
    .replace(/\u6253\u5361$/u, '')
    .replace(/\u89c2\u5149$/u, '')
    .replace(/\u6162\u6e38$/u, '')
    .replace(/\u5165\u4f4f$/u, '')
    .replace(/\u4f11\u95f2$/u, '')
    .replace(/\u8bb2\u89e3$/u, '')
    .replace(/\u540c\u6846$/u, '')
    .trim()

  if (!text || text.length > 8 || GENERIC_TAGS.has(text)) {
    return ''
  }

  return text
}

function mapSpecificTag(text) {
  const value = normalizeText(text)
  if (!value) {
    return ''
  }

  if (value.includes('\u590f\u6cb3') && value.includes('\u7267\u573a')) return '\u590f\u6cb3\u7267\u573a'
  if (value.includes('\u9647\u5357') && containsAny(value, ['\u8336\u4e61', '\u8336\u5c71', '\u8336\u56ed'])) return '\u9647\u5357\u8336\u4e61'
  if (value.includes('\u82e6\u6c34') && value.includes('\u73ab\u7470')) return '\u82e6\u6c34\u73ab\u7470'
  if (value.includes('\u6c38\u767b') && value.includes('\u73ab\u7470')) return '\u6c38\u767b\u73ab\u7470'
  if (value.includes('\u767e\u5408') && containsAny(value, ['\u91c7\u6316', '\u91c7\u6458', '\u79cd\u690d'])) return '\u767e\u5408\u91c7\u6316'
  if (value.includes('\u8349\u8393') && containsAny(value, ['\u91c7\u6458', '\u679c\u56ed', '\u6e29\u5ba4'])) return '\u8349\u8393\u91c7\u6458'
  if (value.includes('\u68a8\u56ed')) return '\u68a8\u56ed\u91c7\u6458'
```

```
  if (value.includes('\u679c\u56ed') && value.includes('\u91c7\u6458')) return '\u679c\u56ed\u91c7\u6458'
  if (value.includes('\u4e39\u971e') && value.includes('\u5f92\u6b65')) return '\u4e39\u971e\u5f92\u6b65'
  if (value.includes('\u4e39\u971e')) return '\u4e39\u971e\u89c2\u666f'
  if (value.includes('\u82b1\u7530')) return '\u82b1\u7530\u6253\u5361'
  if (value.includes('\u82b1\u6d77')) return '\u82b1\u6d77\u6444\u5f71'
  if (containsAny(value, ['\u8336\u4e61', '\u8336\u5c71])) return '\u8336\u4e61\u6162\u6e38'
  if (value.includes('\u8336\u56ed')) return '\u8336\u56ed\u624b\u4f5c'
  if (value.includes('\u975e\u9057') && value.includes('\u5de5\u574a')) return '\u975e\u9057\u5de5\u574a'
  if (value.includes('\u53e4\u6751')) return '\u53e4\u6751\u6f2b\u6e38'
  if (value.includes('\u7530\u56ed') && containsAny(value, ['\u6444\u5f71', '\u98ce\u5149', '\u89c2\u666f'])) return '\u7530\u56ed\u89c2\u666f'
  if (value.includes('\u7530\u56ed') && value.includes('\u82b1')) return '\u7530\u56ed\u82b1\u6d77'
  if (value.includes('\u9732\u8425')) return '\u5c71\u91ce\u9732\u8425'
  if (value.includes('\u6c11\u5bbf')) return '\u4e61\u91ce\u6c11\u5bbf'
  if (value.includes('\u624b\u4f5c')) return normalizeHighlightTag(value) || '\u624b\u4f5c\u5de5\u574a'
  if (value.includes('\u91c7\u6458')) return normalizeHighlightTag(value)

  return normalizeHighlightTag(value)
}

function collectSpecificTags(activity, target, excludedList, max) {
  const candidates = normalizeList(activity.highlights)
    .concat([
      normalizeText(activity.locationName),
      normalizeText(activity.title),
      normalizeText(activity.summary),
    ])
    .map(mapSpecificTag)
    .filter(Boolean)

  candidates.forEach((item) => pushCandidate(target, item, excludedList, max))
}

function collectFallbackTags(activity, target, excludedList, max) {
  normalizeList(activity.tags)
    .map(normalizeHighlightTag)
    .filter(Boolean)
    .forEach((item) => pushCandidate(target, item, excludedList, max))
}

function buildActivityCoverTags(activity = {}, options = {}) {
  const maxTotal = options.maxTotal || 6
  const maxExtra = Math.max(0, maxTotal - 1)
  const durationTag = buildDurationTag(activity.days, activity.durationText)
  const excludedList = [durationTag]
    .concat(normalizeList(activity.travelModeTags))
    .concat(normalizeList(activity.playTags))
  const textPool = normalizeList(activity.highlights).concat([
    normalizeText(activity.locationName),
    normalizeText(activity.title),
    normalizeText(activity.summary),
    normalizeText(activity.content),
    normalizeText(activity.detail),
  ])
  const coverTags = []

```

```
  collectServiceTags(activity, coverTags, excludedList, maxExtra)
  collectStayTags(activity, coverTags, excludedList, maxExtra)
  collectFoodTags(textPool, coverTags, excludedList, maxExtra)
  collectSpecificTags(activity, coverTags, excludedList, maxExtra)
  collectFallbackTags(activity, coverTags, excludedList, maxExtra)

  return {
    durationTag,
    tags: coverTags.slice(0, maxExtra),
    combinedTags: [durationTag].concat(coverTags.slice(0, maxExtra)),
  }
}

module.exports = {
  buildActivityCoverTags,
}
```

??: miniprogram/utils/askConversationStore.js

```
const ASK_CONVERSATION_STORAGE_KEY = "askXiaoheConversations"
const MAX_CONVERSATIONS = 20

function normalizeText(value) {
  return String(value || "").trim()
}

function createConversationId() {
  return `conv_${Date.now()}_${Math.floor(Math.random() * 100000)}`
}

function buildConversationTitle(firstQuestion = "") {
  const text = normalizeText(firstQuestion)
  if (!text) return "新的小禾对话"

  if (/亲子/.test(text) && /(农旅|活动|采摘|研学|农场)/.test(text)) {
    return "附近亲子农旅活动推荐"
  }

  if (/(拍照|打卡)/.test(text) && /(景点|地方|景区|哪里)/.test(text)) {
    return "附近拍照打卡景点推荐"
  }

  if (/(景点|景区|古村|花海|草原|观景)/.test(text)) {
    return "附近乡村景点推荐"
  }

  if (/(特产|乡味|伴手礼|礼盒|美食)/.test(text)) {
    return "附近乡味特产推荐"
  }

  if (/(周末|放松|去处)/.test(text)) {
    return "附近周末放松去处推荐"
  }

  const compact = text.replace(/[? ?! !。,， 、 ; ;: :\s]/g, "")
  return compact.slice(0, 14) || "新的小禾对话"
}

function safeRead() {
  try {
    return wx.getStorageSync(ASK_CONVERSATION_STORAGE_KEY) || []
  } catch (error) {
    return []
  }
}

function safeWrite(list) {
  wx.setStorageSync(ASK_CONVERSATION_STORAGE_KEY, Array.isArray(list) ? list : [])
}

function listConversations() {
  return safeRead()
    .filter((item) => item && item.id)
    .sort((a, b) => Number(b.updatedAt || 0) - Number(a.updatedAt || 0))
}

function getConversationById(id) {
  return listConversations().find((item) => item.id === id) || null
}
```

```

function saveConversation(conversation) {
  if (!conversation || !conversation.id) return null

  const current = listConversations().filter((item) => item.id !== conversation.id)
  const next = [
    {
      id: conversation.id,
      title: normalizeText(conversation.title) || "新的小禾对话",
      firstQuestion: normalizeText(conversation.firstQuestion),
      createdAt: conversation.createdAt || Date.now(),
      updatedAt: conversation.updatedAt || Date.now(),
      messages: Array.isArray(conversation.messages) ? conversation.messages : [],
      source: normalizeText(conversation.source),
      skillMode: normalizeText(conversation.skillMode),
      genericPreferences: conversation.genericPreferences || {},
      currentTaskState: conversation.currentTaskState || null
    }
  ].concat(current).slice(0, MAX_CONVERSATIONS)

  safeWrite(next)
  return next[0]
}

function removeConversation(id) {
  const next = listConversations().filter((item) => item.id !== id)
  safeWrite(next)
  return next
}

function clearConversations() {
  safeWrite([])
}

module.exports = {
  ASK_CONVERSATION_STORAGE_KEY,
  buildConversationTitle,
  clearConversations,
  createConversationId,
  getConversationById,
  listConversations,
  removeConversation,
  saveConversation
}
```

??: miniprogram/utils/collectionStore.js

```
const FAVORITES_KEY = 'unifiedFavorites'
const FOOTPRINTS_KEY = 'unifiedFootprints'

const CATEGORY_ALL = 'all'
const CITY_ALL = 'all'

const TYPE_CONFIG = {
  activity: {
    label: '活动',
    path: '/pages/activityDetail/activityDetail',
  },
  product: {
    label: '商品',
    path: '/pages/productDetail/productDetail',
  },
  scenic: {
    label: '景点',
    path: '/pages/scenicDetail/scenicDetail',
  },
  hotel: {
    label: '酒店/民宿',
    path: '/pages/hotelDetail/hotelDetail',
  },
  knowledge: {
    label: '推文',
    path: '/pages/knowledgeDetail/knowledgeDetail',
  },
}

function safeList(value) {
  return Array.isArray(value) ? value : []
}

function getStorageList(key) {
  try {
    return safeList(wx.getStorageSync(key))
  } catch (error) {
    console.error('[collectionStore] getStorageList failed', key, error)
    return []
  }
}

function setStorageList(key, list) {
  try {
    wx.setStorageSync(key, safeList(list))
  } catch (error) {
    console.error('[collectionStore] setStorageList failed', key, error)
  }
}

function sortByLatest(list, fieldName) {
  return safeList(list).sort((left, right) => Number(right[fieldName] || 0) - Number(left[fieldName] || 0))
}

function buildKey(type, id) {
  return `${type || ''}::${id || ''}`
}

function createTargetUrl(type, id) {
```

```
  const config = TYPE_CONFIG[type] || {}
  return config.path && id ? `${config.path}?id=${id}` : ''
}

function normalizeText(value = '') {
  return String(value || '').trim()
}

function normalizeRecord(input = {}) {
  const type = normalizeText(input.type)
  const id = normalizeText(input.id)
  const config = TYPE_CONFIG[type] || {}
  const key = buildKey(type, id)
  const normalizedBadgeText = normalizeText(input.badgeText)
  const badgeText = type === 'knowledge'
    ? config.label
    : (normalizedBadgeText || config.label || '')

  return {
    key,
    type,
    id,
    categoryLabel: config.label || '',
    title: normalizeText(input.title) || '未命名内容',
    cover: normalizeText(input.cover),
    city: normalizeText(input.city) || '未知城市',
    regionText: normalizeText(input.regionText),
    summary: normalizeText(input.summary),
    priceText: normalizeText(input.priceText),
    metaText: normalizeText(input.metaText),
    badgeText,
    author: normalizeText(input.author),
    statsText: normalizeText(input.statsText),
    scoreText: normalizeText(input.scoreText),
    targetUrl: normalizeText(input.targetUrl) || createTargetUrl(type, id),
    updatedAt: Number(input.updatedAt || 0),
    favoritedAt: Number(input.favoritedAt || 0),
    visitedAt: Number(input.visitedAt || 0),
  }
}

function getFavorites() {
  return sortByLatest(
    getStorageList(FAVORITES_KEY).map((item) => normalizeRecord(item)),
    'favoritedAt'
  )
}

function getFootprints() {
  return sortByLatest(
    getStorageList(FOOTPRINTS_KEY).map((item) => normalizeRecord(item)),
    'visitedAt'
  )
}

function isFavorited(type, id) {
  const targetKey = buildKey(type, id)
  return getFavorites().some((item) => item.key === targetKey)
}

function toggleFavorite(record = {}) {
  const normalized = normalizeRecord(record)
```

```
  const currentList = getFavorites()
  const exists = currentList.some((item) => item.key === normalized.key)

  if (exists) {
    const nextList = currentList.filter((item) => item.key !== normalized.key)
    setStorageList(FAVORITES_KEY, nextList)
    return false
  }

  const nextItem = {
    ...normalized,
    favoritedAt: Date.now(),
    updatedAt: Date.now(),
  }
  const nextList = [nextItem, ...currentList.filter((item) => item.key !== normalized.key)]
  setStorageList(FAVORITES_KEY, nextList)
  return true
}

function recordFootprint(record = {}) {
  const normalized = normalizeRecord(record)
  if (!normalized.type || !normalized.id) {
    return
  }

  const currentList = getFootprints().filter((item) => item.key !== normalized.key)
  const nextItem = {
    ...normalized,
    visitedAt: Date.now(),
    updatedAt: Date.now(),
  }
  setStorageList(FOOTPRINTS_KEY, [nextItem, ...currentList])
}

function clearFootprints() {
  setStorageList(FOOTPRINTS_KEY, [])
}

function deleteFavorites(keys = []) {
  const keySet = new Set(safeList(keys).filter(Boolean))
  if (!keySet.size) {
    return
  }

  const nextList = getFavorites().filter((item) => !keySet.has(item.key))
  setStorageList(FAVORITES_KEY, nextList)
}

module.exports = {
  CATEGORY_ALL,
  CITY_ALL,
  TYPE_CONFIG,
  getFavorites,
  getFootprints,
  isFavorited,
  toggleFavorite,
  deleteFavorites,
  recordFootprint,
  clearFootprints,
}
```

??: miniprogram/utils/knowledgeArticle.js

```
const { resolveMediaSource } = require('./mediaAssets')

const CLOUD_ENV_ID = 'cloud1-3ghmr5ki7b1172fe.636c-cloud1-3ghmr5ki7b1172fe-1403917845'
const CLOUD_KNOWLEDGE_COVER_PREFIX = `cloud://${CLOUD_ENV_ID}/knowledge/covers`
const KNOWLEDGE_COVER_FILE_MAP = {
  'agri-001': 'spring-vegetable.png',
  'agri-002': 'village-chicken.png',
  'agri-003': 'corn-disease.png',
  'agri-004': 'bee-guide.png',
  'agri-005': 'rice-brand.png',
  'culture-001': 'summer-palace.png',
  'culture-002': 'intangible-heritage.png',
  'culture-003': 'folk-custom.png',
  'culture-004': 'travel-tips.png',
}

function normalizeText(value = '') {
  return String(value || '').trim()
}

function normalizeArray(value) {
  return Array.isArray(value) ? value : []
}

function normalizeContentType(type = '') {
  const value = normalizeText(type).toLowerCase()
  if (value === 'h2' || value === 'heading') {
    return 'heading'
  }
  if (value === 'img' || value === 'image') {
    return 'image'
  }
  return 'paragraph'
}

function getFileName(value = '') {
  const normalized = normalizeText(value)
  if (!normalized) {
    return ''
  }

  const segments = normalized.split('/')
  return normalizeText(segments[segments.length - 1])
}

function buildKnowledgeCoverFileID(fileName = '') {
  const normalized = getFileName(fileName)
  return normalized ? `${CLOUD_KNOWLEDGE_COVER_PREFIX}/${normalized}` : ''
}

function normalizeKnowledgeCover(article = {}) {
  const cover = normalizeText(article.cover)
  if (/^(cloud|https?):\/\//i.test(cover)) {
    return cover
  }

  const coverFileName = getFileName(cover) || KNOWLEDGE_COVER_FILE_MAP[normalizeText(article.id)]
  return buildKnowledgeCoverFileID(coverFileName)
}
```

```

async function resolveKnowledgeArticleMedia(article = {}) {
  const content = await Promise.all(
    normalizeArray(article.content).map(async (item = {}) => {
      const type = normalizeContentType(item.type)
      if (type === 'image') {
        const src = await resolveMediaSource(item.src || item.image || '', '')
        return {
          ...item,
          type,
          src,
        }
      }

      return {
        ...item,
        type,
        text: normalizeText(item.text),
      }
    })
  )

  return {
    ...article,
    tags: normalizeArray(article.tags).filter(Boolean),
    cover: await resolveMediaSource(normalizeKnowledgeCover(article), ''),
    content,
  }
}

module.exports = {
  normalizeText,
  normalizeArray,
  normalizeContentType,
  normalizeKnowledgeCover,
  resolveKnowledgeArticleMedia,
}
```

??: miniprogram/utils/mediaAssets.js

```
const ACTIVITY_MEDIA_MAP = {
  'lz-yuzhong-strawberry-family-day': {
    aliases: ['榆中', '草莓', '亲子'],
    localCover: '/images/activities/lz-yuzhong-strawberry-family-day.jpg',
  },
  'lz-gaolan-country-photo-day': {
    aliases: ['皋兰', '田园', '摄影'],
    localCover: '/images/activities/lz-gaolan-country-photo-day.jpg',
  },
  'lz-yongdeng-rose-weekend': {
    aliases: ['永登', '玫瑰', '周末'],
    localCover: '/images/activities/lz-yongdeng-rose-weekend.jpg',
  },
  'lz-suburb-farm-study-camp': {
    aliases: ['近郊', '农事', '研学'],
    localCover: '/images/activities/lz-suburb-farm-study-camp.jpg',
  },
  'lz-bailihe-handmade-food-tour': {
    aliases: ['百合', '乡味', '手作'],
    localCover: '/images/activities/lz-bailihe-handmade-food-tour.jpg',
  },
  'lz-kushui-rose-culture-day': {
    aliases: ['苦水玫瑰', '玫瑰季', '非遗文化'],
    localCover: '/images/nav-academy.png',
  },
  'lz-kushui-danxia-hike-day': {
    aliases: ['苦水丹霞', '红色地貌', '轻徒步'],
    localCover: '/images/nav-academy.png',
  },
  'lz-xigu-baihe-culture-day': {
    aliases: ['西果园', '百合田园', '百合文化'],
    localCover: '/images/nav-academy.png',
  },
  'lz-gaolan-shichuan-pear-garden-day': {
    aliases: ['什川', '古梨园', '皋兰'],
    localCover: '/images/nav-academy.png',
  },
  'lz-qilihe-shenjialing-red-green-fusion': {
    aliases: ['沈家岭', '红绿融合', '红色研学'],
    localCover: '/images/nav-academy.png',
  },
  'lz-bali-huazhaizi-peach-garden': {
    aliases: ['花寨子', '世外桃园', '鲜桃采摘'],
    localCover: '/images/nav-academy.png',
  },
  'tianshui-qinan-fruit-pick-day': {
    aliases: ['天水', '秦安', '果园'],
    localCover: '/images/activities/tianshui-qinan-fruit-pick-day.jpg',
  },
  'zhangye-rural-homestay-weekend': {
    aliases: ['张掖', '民宿', '乡野'],
    localCover: '/images/activities/zhangye-rural-homestay-weekend.jpg',
  },
  'longnan-tea-hill-folk-experience': {
    aliases: ['陇南', '茶山', '民俗'],
    localCover: '/images/activities/longnan-tea-hill-folk-experience.jpg',
  },
  'linxia-food-culture-village-tour': {
    aliases: ['临夏', '美食', '文化'],
    localCover: '/images/activities/linxia-food-culture-village-tour.jpg',
```

```
  },
  'gannan-ranch-life-light-tour': {
    aliases: ['甘南', '草原', '牧场'],
    localCover: '/images/activities/gannan-ranch-life-light-tour.jpg',
    cloudCover: 'cloud://cloud1-3ghmr5ki7b1172fe.636c-cloud1-3ghmr5ki7b1172fe-1403917845/activities/covers/gannan-ranch-life-light-tour.jpg',
    cloudBanner: 'cloud://cloud1-3ghmr5ki7b1172fe.636c-cloud1-3ghmr5ki7b1172fe-1403917845/activities/banners/gannan-ranch-life-light-tour.jpg',
    cloudGallery: [
      'cloud://cloud1-3ghmr5ki7b1172fe.636c-cloud1-3ghmr5ki7b1172fe-1403917845/activities/gallery/gannan-ranch-life-light-tour-1.jpg',
      'cloud://cloud1-3ghmr5ki7b1172fe.636c-cloud1-3ghmr5ki7b1172fe-1403917845/activities/gallery/gannan-ranch-life-light-tour-2.jpg',
    ],
  },
}

const DEFAULT_ACTIVITY_KEY = 'lz-yuzhong-strawberry-family-day'
const CLOUD_FILE_CACHE = Object.create(null)
const CLOUD_ACTIVITY_PREFIX = 'cloud://cloud1-3ghmr5ki7b1172fe.636c-cloud1-3ghmr5ki7b1172fe-1403917845/activities'
const MAX_GALLERY_COUNT = 6

function normalizeText(value = '') {
  return String(value || '').trim()
}

function isLocalMedia(src = '') {
  return normalizeText(src).startsWith('/images/')
}

function isRemoteMedia(src = '') {
  return /^https?:\/\//i.test(normalizeText(src))
}

function isCloudFileId(src = '') {
  return /^cloud:\/\//i.test(normalizeText(src))
}

function isRenderableMedia(src = '') {
  return isLocalMedia(src) || isRemoteMedia(src)
}

function findActivityKey(activity = {}) {
  const seedKey = normalizeText(activity.seedKey)
  if (seedKey && ACTIVITY_MEDIA_MAP[seedKey]) {
    return seedKey
  }

  const title = normalizeText(activity.title)
  if (!title) {
    return DEFAULT_ACTIVITY_KEY
  }

  const matchedKey = Object.keys(ACTIVITY_MEDIA_MAP).find((key) => {
    const aliases = ACTIVITY_MEDIA_MAP[key].aliases || []
    return aliases.some((alias) => title.includes(alias))
  })

  return matchedKey || DEFAULT_ACTIVITY_KEY
}

```

```
      },
      weather: null,
      answer: `我已经识别到你想问的大概地区是${resolved.label || "目标地区"}，但位置坐标还没拿到，所以实时天气暂不可用。你可以换的区县名再试一次。`,
    };
  }

  try {
    const weatherData = await fetchOpenMeteoWeather(resolved.latitude, resolved.longitude);
    const locationLabel = normalizeText(resolved.label) || "你当前所在地区";
    const current = weatherData?.current || {};
    const daily = weatherData?.daily || {};

    return {
      success: true,
      subType: "weather",
      resolvedLocation: {
        label: normalizeText(resolved.label),
        province: normalizeText(resolved.province),
        city: normalizeText(resolved.city),
        district: normalizeText(resolved.district),
        adcode: normalizeText(resolved.adcode),
        latitude: normalizeText(resolved.latitude),
        longitude: normalizeText(resolved.longitude),
      },
      weather: {
        weatherText: mapWeatherCode(current.weather_code),
        temperature: formatNumber(current.temperature_2m),
        apparentTemperature: formatNumber(current.apparent_temperature),
        minTemperature: Array.isArray(daily.temperature_2m_min) ? formatNumber(daily.temperature_2m_min[0]) : "",
        maxTemperature: Array.isArray(daily.temperature_2m_max) ? formatNumber(daily.temperature_2m_max[0]) : "",
        precipitationProbability: Array.isArray(daily.precipitation_probability_max)
          ? formatNumber(daily.precipitation_probability_max[0])
          : "",
        windSpeed: formatNumber(current.wind_speed_10m),
      },
      answer: buildWeatherAnswer(locationLabel, weatherData),
    };
  } catch (error) {
    return {
      success: false,
      subType: "weather",
      resolvedLocation: {
        label: normalizeText(resolved.label),
        province: normalizeText(resolved.province),
        city: normalizeText(resolved.city),
        district: normalizeText(resolved.district),
        adcode: normalizeText(resolved.adcode),
        latitude: normalizeText(resolved.latitude),
        longitude: normalizeText(resolved.longitude),
      },
      weather: null,
      answer: "实时天气暂不可用，刚才查询天气服务时失败了。你可以稍后再试。",
    };
  }
}

export async function buildLocationQueryResult({
  contextPayload = {},
```

```
} = {}) {
  const resolved = await resolveLocation(contextPayload, "");
  if (!resolved) {
    return {
      success: false,
      subType: "location",
      resolvedLocation: null,
      answer: "当前位置信息暂时不可用，请稍后再试。你可以先开启定位权限，或者直接告诉我你所在的城市。",
    };
  }

  const label =
    normalizeText(resolved.label) ||
    [resolved.province, resolved.city, resolved.district].filter(Boolean).join("") ||
    "你当前所在地区";

  return {
    success: true,
    subType: "location",
    resolvedLocation: {
      label: normalizeText(label),
      province: normalizeText(resolved.province),
      city: normalizeText(resolved.city),
      district: normalizeText(resolved.district),
      adcode: normalizeText(resolved.adcode),
      latitude: normalizeText(resolved.latitude),
      longitude: normalizeText(resolved.longitude),
    },
    answer: `我目前判断你大概在${label}。如果你想，我也可以继续帮你查这个地区的实时天气。`,
  };
}

function buildBuddyMissingFieldResponse(workflow = {}) {
  return workflow?.missingField?.ask || "我先帮你把这次找搭子的信息收清楚，你更想找什么样的同伴？";
}

function buildBuddyNoCandidateResponse() {
  return "当前没有合适候选。你可以补充一下时间、出发地或同行偏好，我再继续帮你缩小范围。";
}

function buildGuideMissingFieldResponse(workflow = {}) {
  return workflow?.missingField?.ask || "我先帮你把这次攻略信息收清楚，你打算什么时候去？";
}

function buildFeedbackDirectResponse(workflow = {}) {
  const feedbackType = workflow?.feedbackType || "产品反馈";
  if (feedbackType === "情绪倾诉") {
    return "我接住你现在这份情绪。你愿意的话，可以再告诉我刚刚最让你难受的点是什么，我会认真听。";
  }

  if (feedbackType === "平台建议") {
    return "这条建议我记下了。你要是愿意，可以再补一句你最希望它先改哪一处，我好把重点收得更准。";
  }

  return "我收到你的反馈了。要是你愿意，可以再补一点具体场景或步骤，这样我能更准确地整理问题。";
}

export async function buildDirectAnswer({
  question = "",
  contextPayload = {},
  cloudbaseUserId = "",
} = {}) {
```

```
  const route = routeIntent(question);
  if (route.intent === "weather") {
    return buildWeatherDirectResponse(question, contextPayload);
  }
  if (route.intent === "where_am_i") {
    return buildWhereAmIDirectResponse(contextPayload);
  }

  const mainline = detectMainline(question, contextPayload);
  const workflow = await buildConversationWorkflow({
    question,
    contextPayload,
    cloudbaseUserId,
  });

  console.log("[agent-yuxiaohe] buildDirectAnswer", {
    question: normalizeText(question),
    mainline,
    isReady: workflow?.isReady,
    missingField: workflow?.missingField?.key || workflow?.missingField?.label || "",
    collected: workflow?.fields || {},
    feedbackType: workflow?.feedbackType || "",
  });

  if (mainline === MAINLINE.BUDDY) {
    if (!workflow.isReady) {
      return buildBuddyMissingFieldResponse(workflow);
    }

    if (!workflow.buddyCandidates?.length) {
      return buildBuddyNoCandidateResponse();
    }

    return "";
  }

  if (mainline === MAINLINE.GUIDE && !workflow.isReady) {
    return buildGuideMissingFieldResponse(workflow);
  }

  if (mainline === MAINLINE.FEEDBACK && normalizeText(question).length <= 18) {
    return buildFeedbackDirectResponse(workflow);
  }

  return "";
}
```

??: cloudfunctions/agent-yuxiaohe-1grmumm967563411/src/utils.js
---

```
import { jwtDecode } from "jwt-decode";
import tcb from "@cloudbase/node-sdk";

const LOCATION_WORD_REGEXP = /(附近|周边|本地|同城|这里|这边|当前位置|现在)/u;
const DEFAULT_REGION_LABEL = "未提供地区";
const MAX_DATASET_FETCH = 60;
const MAX_CANDIDATE_PER_TYPE = 3;
const BUDDY_MAX_CANDIDATES = 3;
const GUIDE_MAX_CANDIDATES = 6;

export const MAINLINE = {
  BUDDY: "buddy_matching",
  GUIDE: "guide_customization",
  WEATHER: "weather_location",
  FEEDBACK: "xiaohe_feedback",
  GENERIC: "generic_guard",
};

const MAINLINE_SKILL_MODE_MAP = {
  buddy_matching: MAINLINE.BUDDY,
  guide_customization: MAINLINE.GUIDE,
  xiaohe_feedback: MAINLINE.FEEDBACK,
};

const BUDDY_KEYWORDS = ["搭子", "同行", "一起去", "找人一起", "结伴", "拼玩", "拼车", "一起玩", "一起旅游", "一起旅行"];
const GUIDE_KEYWORDS = ["攻略", "路线", "行程", "怎么玩", "安排", "推荐路线", "定制", "规划"];
const WEATHER_KEYWORDS = [
  "天气",
  "气温",
  "温度",
  "下雨",
  "降雨",
  "穿什么",
  "穿啥",
  "位置",
  "定位",
  "我在哪",
  "附近天气",
];
const FEEDBACK_KEYWORDS = [
  "吐槽",
  "反馈",
  "建议",
  "意见",
  "抱怨",
  "难用",
  "卡顿",
  "bug",
  "心情",
  "烦",
  "难受",
  "委屈",
  "emo",
];

let cloudbaseApp = null;

export function normalizeText(value) {
  return String(value || "").trim();
```

```
}

function normalizeTextLower(value) {
  return normalizeText(value).toLowerCase();
}

function normalizeArray(value) {
  return Array.isArray(value)
    ? value.map((item) => normalizeText(item)).filter(Boolean)
    : [];
}

function uniqueList(list = []) {
  return Array.from(new Set((list || []).filter(Boolean)));
}

function maskIdentifier(value = "") {
  const normalized = normalizeText(value);
  if (!normalized) return "";
  if (normalized.length <= 8) return normalized;
  return `${normalized.slice(0, 4)}***${normalized.slice(-4)}`;
}

function includesAny(text = "", keywords = []) {
  return keywords.some((keyword) => text.includes(keyword));
}

function isBuddyIntentText(text = "") {
  const normalized = normalizeText(text);
  if (!normalized) return false;
  if (includesAny(normalized, BUDDY_KEYWORDS)) return true;

  return (
    /(找|想找|帮我找).*(一起|结伴|同行)/u.test(normalized) ||
    /(一起).*(找|想找|帮我找)/u.test(normalized) ||
    /(找|想找|帮我找).*(女生|男生|小姐姐|小哥哥)/u.test(normalized) ||
    /(女生|男生|小姐姐|小哥哥).*(一起|同行|结伴)/u.test(normalized)
  );
}

function removeRegionSuffix(value) {
  return normalizeText(value)
    .replace(/(特别行政区|自治区|自治州|地区)$/u, "")
    .replace(/(省|市|区|县|旗)$/u, "");
}

function buildRegionTokens(location = {}) {
  return uniqueList(
    [
      location.province,
      location.city,
      location.district,
      location.displayName,
      location.locationText,
    ]
      .map(removeRegionSuffix)
      .filter(Boolean)
  );
}

function buildExplicitRegionTokens(region = "") {
  const normalizedRegion = normalizeText(region);
```

```
  if (!normalizedRegion) return [];

  return uniqueList(
    [normalizedRegion, removeRegionSuffix(normalizedRegion)].filter(Boolean)
  );
}

function buildRegionLabel(location = {}) {
  const displayName = normalizeText(location.displayName);
  if (displayName) {
    return displayName;
  }

  const region = [location.province, location.city, location.district]
    .map(removeRegionSuffix)
    .filter(Boolean)
    .join(" / ");

  return region || normalizeText(location.locationText) || DEFAULT_REGION_LABEL;
}

function getConversationHistory(contextPayload = {}) {
  if (!Array.isArray(contextPayload?.history)) {
    return [];
  }

  return contextPayload.history
    .map((item) => ({
      role: item?.role === "assistant" || item?.role === "ai" ? "assistant" : "user",
      text: normalizeText(item?.text || item?.content || ""),
    }))
    .filter((item) => item.text)
    .slice(-20);
}

function inferFieldFromAssistant(mainline = "", assistantText = "") {
  const text = normalizeText(assistantText);
  if (!text) return "";

  if (mainline === MAINLINE.GUIDE) {
    if (text.includes("最想去哪里")) return "destination";
    if (text.includes("什么时候去")) return "time";
    if (text.includes("几个人一起")) return "peopleCount";
    if (text.includes("什么关系")) return "relationship";
    if (text.includes("预算")) return "budget";
  }

  if (mainline === MAINLINE.BUDDY) {
    if (text.includes("从哪里出发")) return "departure";
    if (text.includes("最想去哪里")) return "destination";
    if (text.includes("什么时候出发")) return "time";
    if (text.includes("同行搭子")) return "companionPreference";
  }

  return "";
}

function inferMainlineFromHistory(contextPayload = {}) {
  const history = getConversationHistory(contextPayload);
  if (!history.length) return "";

  const lastAssistantText =
```

```
    [...history].reverse().find((item) => item.role === "assistant")?.text || "";
  const combinedText = history.map((item) => item.text).join("\n");

  if (
    lastAssistantText.includes("最想去哪里") ||
    lastAssistantText.includes("什么时候去") ||
    lastAssistantText.includes("几个人一起") ||
    lastAssistantText.includes("什么关系") ||
    lastAssistantText.includes("预算")
  ) {
    return MAINLINE.GUIDE;
  }

  if (
    lastAssistantText.includes("从哪里出发") ||
    lastAssistantText.includes("什么时候出发") ||
    lastAssistantText.includes("同行搭子")
  ) {
    return MAINLINE.BUDDY;
  }

  if (
    lastAssistantText.includes("我会认真听") ||
    lastAssistantText.includes("收到你的反馈") ||
    lastAssistantText.includes("收到这条建议") ||
    lastAssistantText.includes("最让你难受的点")
  ) {
    return MAINLINE.FEEDBACK;
  }

  if (includesAny(combinedText, GUIDE_KEYWORDS)) return MAINLINE.GUIDE;
  if (isBuddyIntentText(combinedText)) return MAINLINE.BUDDY;
  if (includesAny(combinedText, FEEDBACK_KEYWORDS)) return MAINLINE.FEEDBACK;
  if (includesAny(combinedText, WEATHER_KEYWORDS)) return MAINLINE.WEATHER;

  return "";
}

function inferTaskStateFromHistory(contextPayload = {}) {
  const explicitState = contextPayload?.currentTaskState || {};
  const mainline =
    normalizeText(explicitState?.mainline) || inferMainlineFromHistory(contextPayload);

  if (!mainline) {
    return explicitState || {};
  }

  const history = getConversationHistory(contextPayload);
  const inferredState = {
    ...explicitState,
    mainline,
    collected: {
      ...(explicitState?.collected || {}),
    },
    lastAskedField: normalizeText(explicitState?.lastAskedField),
    feedbackType: normalizeText(explicitState?.feedbackType),
  };

  const userTexts = history
    .filter((item) => item.role === "user")
    .map((item) => item.text);

```

```
  if (mainline === MAINLINE.GUIDE) {
    const collected = { ...(inferredState.collected || {}) };
    const historyInferenceContext = {
      ...(contextPayload || {}),
      disableHistoryInference: true,
      currentTaskState: {
        ...(explicitState || {}),
        mainline,
        collected,
      },
      skillContext: {
        ...(contextPayload?.skillContext || {}),
        collected: {
          ...(contextPayload?.skillContext?.collected || {}),
          ...collected,
        },
      },
    };

    userTexts.forEach((text) => {
      historyInferenceContext.currentTaskState.collected = collected;
      historyInferenceContext.skillContext.collected = {
        ...(contextPayload?.skillContext?.collected || {}),
        ...collected,
      };

      if (!collected.time) collected.time = extractTime(text, historyInferenceContext);
      if (!collected.peopleCount) {
        collected.peopleCount = extractCount(
          text,
          historyInferenceContext,
          ["peopleCount"],
          /([一二三四五六七八九十两\d]+(?:人|位))/u
        );
      }
      if (!collected.relationship) {
        collected.relationship = extractRelationship(text, historyInferenceContext);
      }
      if (!collected.peopleCount) {
        collected.peopleCount = normalizePeopleCountFromRelationship(
          collected.relationship,
          collected.peopleCount
        );
      }
      if (!collected.budget) collected.budget = extractGuideBudgetValue(text, historyInferenceContext);
      if (!collected.destination) collected.destination = extractDestination(text, historyInferenceContext);
    });
    inferredState.collected = collected;
  }

  if (mainline === MAINLINE.BUDDY) {
    const collected = { ...(inferredState.collected || {}) };
    const historyInferenceContext = {
      ...(contextPayload || {}),
      disableHistoryInference: true,
      currentTaskState: {
        ...(explicitState || {}),
        mainline,
        collected,
      },
```

```
      skillContext: {
        ...(contextPayload?.skillContext || {}),
        collected: {
          ...(contextPayload?.skillContext?.collected || {}),
          ...collected,
        },
      },
    };

    userTexts.forEach((text) => {
      historyInferenceContext.currentTaskState.collected = collected;
      historyInferenceContext.skillContext.collected = {
        ...(contextPayload?.skillContext?.collected || {}),
        ...collected,
      };

      if (!collected.departure) {
        const location = contextPayload?.location || {};
        collected.departure = normalizeText(location.city || location.displayName);
      }
      if (!collected.destination) collected.destination = extractDestination(text, historyInferenceContext);
      if (!collected.time) collected.time = extractTime(text, historyInferenceContext);
      if (!collected.companionPreference) {
        collected.companionPreference = extractCompanionPreference(text, historyInferenceContext);
      }
    });
    inferredState.collected = collected;
  }

  if (mainline === MAINLINE.FEEDBACK && !inferredState.feedbackType) {
    const latestUserText = [...userTexts].reverse().find(Boolean) || "";
    inferredState.feedbackType = classifyFeedbackType(latestUserText);
  }

  if (!inferredState.lastAskedField) {
    const lastAssistantText =
      [...history].reverse().find((item) => item.role === "assistant")?.text || "";
    inferredState.lastAskedField = inferFieldFromAssistant(mainline, lastAssistantText);
  }

  return inferredState;
}

function getCurrentTaskState(contextPayload = {}) {
  if (contextPayload?.disableHistoryInference) {
    return contextPayload?.currentTaskState || {};
  }

  return inferTaskStateFromHistory(contextPayload);
}

function getMergedCollected(contextPayload = {}) {
  const skillCollected = contextPayload?.skillContext?.collected || {};
  const taskCollected = getCurrentTaskState(contextPayload)?.collected || {};

  return {
    ...skillCollected,
    ...taskCollected,
  };
}
```

```

function readStateValue(contextPayload = {}, keys = []) {
  const collected = getMergedCollected(contextPayload);
  for (const key of keys) {
    const value = normalizeText(collected?.[key]);
    if (value) return value;
  }
  return "";
}

function mergeSkillLocation(location = {}, contextPayload = {}) {
  const skillMode = normalizeText(contextPayload?.skillContext?.mode);
  const collected = getMergedCollected(contextPayload);
  const explicitRegion =
    skillMode === MAINLINE.GUIDE
      ? normalizeText(collected.destination || collected.region)
      : normalizeText(collected.region);

  if (!explicitRegion) {
    return location || {};
  }

  return {
    ...(location || {}),
    displayName: explicitRegion,
    locationText: explicitRegion,
  };
}

function safeNumber(value) {
  const number = Number(value);
  return Number.isFinite(number) ? number : 0;
}

function getPriceText(item = {}, type = "activity") {
  const price = safeNumber(item.priceFrom || item.price || 0);
  if (!price) {
    return type === "hotel" ? "价格待定" : "可咨询";
  }
  return type === "hotel" ? `□${price}/晚` : `□${price}起`;
}

function getItemRegionText(item = {}) {
  return [item.province, item.city, item.district]
    .map(removeRegionSuffix)
    .filter(Boolean)
    .join(" / ");
}

function joinSearchText(item = {}) {
  return normalizeTextLower(
    [
      item.title,
      item.name,
      item.summary,
      item.content,
      item.detail,
      item.desc,
      item.description,
      item.locationName,
      item.address,
      item.province,
```

```
      item.city,
      item.district,
      item.transport,
      item.stay,
      ...normalizeArray(item.tags),
      ...normalizeArray(item.travelModeTags),
      ...normalizeArray(item.playTags),
      ...normalizeArray(item.suitableGroups),
      ...normalizeArray(item.highlights),
      ...normalizeArray(item.itinerary),
      ...normalizeArray(item.categoryTags),
      ...normalizeArray(item.dnaTags),
    ].join(" ")
  );
}

function buildIntentTokens(question = "", userProfile = {}) {
  const source = normalizeText(question);
  const tokens = [];
  const keywordMap = [
    "亲子",
    "农旅",
    "活动",
    "景点",
    "民宿",
    "酒店",
    "住宿",
    "商品",
    "特产",
    "采摘",
    "研学",
    "手作",
    "花海",
    "周末",
    "拍照",
    "露营",
    "慢游",
    "乡村",
    "农场",
    "萌宠",
  ];

  keywordMap.forEach((keyword) => {
    if (source.includes(keyword)) {
      tokens.push(keyword);
    }
  });

  normalizeArray(userProfile.dnaTags).forEach((tag) => tokens.push(tag));
  return uniqueList(tokens);
}

function detectRequestedTypes(question = "") {
  const text = normalizeText(question);
  const requested = [];

  if (/活动|采摘|研学|体验|农旅|亲子|周末|玩/u.test(text)) {
    requested.push("activity");
  }
  if (/景点|观光|古村|花海|打卡|拍照/u.test(text)) {
    requested.push("scenic");
  }
```

```
  if (/民宿|酒店|住宿|住/u.test(text)) {
    requested.push("hotel");
  }
  if (/特产|商品|礼盒|伴手礼/u.test(text)) {
    requested.push("product");
  }

  return requested.length ? requested : ["activity", "scenic", "hotel", "product"];
}

function scoreLocation(item = {}, regionTokens = []) {
  if (!regionTokens.length) return 0;

  const text = joinSearchText(item);
  let score = 0;
  regionTokens.forEach((token, index) => {
    if (token && text.includes(normalizeTextLower(token))) {
      score += 15 - Math.min(index * 2, 8);
    }
  });
  return score;
}

function scoreIntent(item = {}, intentTokens = []) {
  if (!intentTokens.length) return 0;

  const text = joinSearchText(item);
  return intentTokens.reduce((score, token) => {
    if (!token) return score;
    return score + (text.includes(normalizeTextLower(token)) ? 8 : 0);
  }, 0);
}

function scoreNearbyIntent(question = "", item = {}, location = {}) {
  if (!LOCATION_WORD_REGEXP.test(question)) return 0;

  const regionText = buildRegionLabel(location);
  const itemRegion =
    getItemRegionText(item) || normalizeText(item.locationName) || normalizeText(item.address);

  return itemRegion && regionText !== DEFAULT_REGION_LABEL && itemRegion.includes(removeRegionSuffix(regionText))
    ? 10
    : 0;
}

function scoreSourceType(type, requestedTypes = []) {
  if (!requestedTypes.length) return 0;
  if (requestedTypes[0] === type) return 20;
  return requestedTypes.includes(type) ? 10 : -10;
}

function sortByScore(list = []) {
  return [...list].sort((a, b) => {
    if (b.score !== a.score) {
      return b.score - a.score;
    }

    return safeNumber(b.raw.updatedAt || b.raw.createTime || 0) - safeNumber(a.raw.updatedAt || a.raw.createTime || 0);
  });
```

```
549 }
550
551 function summarizeItem(type, item = {}) {
552   const title = normalizeText(
553     item.title ||
554       item.name ||
555       item.locationName ||
556       {
557         activity: "农旅活动",
558         scenic: "乡村景点",
559         hotel: "乡野民宿",
560         product: "乡味商品",
561       }[type]
562   );
563
564   return {
565     id: normalizeText(item._id || item.id),
566     type,
567     title,
568     region: getItemRegionText(item) || normalizeText(item.locationName) || normalizeText
    (item.address) || "地区待补充",
569     priceText: getPriceText(item, type),
570     summary: normalizeText(item.summary || item.content || item.desc || item.description
     || item.detail),
571     tags: uniqueList([
572       ...normalizeArray(item.tags),
573       ...normalizeArray(item.travelModeTags),
574       ...normalizeArray(item.playTags),
575       ...normalizeArray(item.suitableGroups),
576       ...normalizeArray(item.categoryTags),
577     ]).slice(0, 4),
578   };
579 }
580
581 function formatCandidateLine(candidate, index) {
582   const summary = candidate.summary ? `; 简介：${candidate.summary}` : "";
583   const tags = candidate.tags.length ? `; 标签：${candidate.tags.join("、")}` : "";
584   return `${index + 1}. [${candidate.type}] ${candidate.title} (ID: ${candidate.id || "无"
    }; 地区：${candidate.region}; 价格：${candidate.priceText}${tags}${summary}) `;
585 }
586
587 function itemMatchesExplicitRegion(item = {}, regionTokens = []) {
588   if (!regionTokens.length) return true;
589
590   const text = joinSearchText(item);
591   return regionTokens.some((token) => text.includes(normalizeTextLower(token)));
592 }
593
594 function getCloudbaseApp() {
595   if (cloudbaseApp) return cloudbaseApp;
596
597   const env = process.env.TCB_ENV || process.env.ENV_ID || "";
598   cloudbaseApp = tcb.init({
599     env,
600     secretId: process.env.TENCENTCLOUD_SECRETID || "",
601     secretKey: process.env.TENCENTCLOUD_SECRETKEY || "",
602     sessionToken: process.env.TENCENTCLOUD_SESSIONTOKEN || "",
603   });
604
605   return cloudbaseApp;
606 }
607
```

```
async function fetchCollection(collectionName, query = null) {
  const app = getCloudbaseApp();
  let request = app.database().collection(collectionName);
  if (query) {
    request = request.where(query);
  }

  const result = await request.limit(MAX_DATASET_FETCH).get();
  return Array.isArray(result?.data) ? result.data : [];
}

async function fetchUserByOpenid(openid = "") {
  const normalizedOpenid = normalizeText(openid);
  if (!normalizedOpenid) return null;

  const result = await fetchCollection("users", { openid: normalizedOpenid });
  console.log("[agent-yuxiaohe] fetchUserByOpenid", {
    requestedOpenid: maskIdentifier(normalizedOpenid),
    matchedCount: result.length,
    matchedByOpenid: result.some(
      (item) => normalizeText(item?.openid) === normalizedOpenid
    ),
    matchedBy_Openid: result.some(
      (item) => normalizeText(item?._openid) === normalizedOpenid
    ),
  });
  return result[0] || null;
}

async function loadPlatformDataset() {
  const [activities, hotels, scenics, products] = await Promise.all([
    fetchCollection("activities", { status: "published" }),
    fetchCollection("hotels", { status: true }),
    fetchCollection("scenics"),
    fetchCollection("products"),
  ]);

  return { activities, hotels, scenics, products };
}

function buildRankedCandidates({
  question = "",
  location = {},
  userProfile = {},
  datasets = {},
  explicitRegion = "",
}) {
  const regionTokens = buildRegionTokens(location);
  const explicitRegionTokens = buildExplicitRegionTokens(explicitRegion);
  const requestedTypes = detectRequestedTypes(question);
  const intentTokens = buildIntentTokens(question, userProfile);

  const candidates = []
    .concat((datasets.activities || []).map((item) => ({ type: "activity", raw: item }))
    .concat((datasets.scenics || []).map((item) => ({ type: "scenic", raw: item })))
    .concat((datasets.hotels || []).map((item) => ({ type: "hotel", raw: item })))
    .concat((datasets.products || []).map((item) => ({ type: "product", raw: item })))
    .map((entry) => {
      const summary = summarizeItem(entry.type, entry.raw);
      const score =
        scoreSourceType(entry.type, requestedTypes) +
```

```
        scoreLocation(entry.raw, regionTokens) +
        scoreIntent(entry.raw, intentTokens) +
        scoreNearbyIntent(question, entry.raw, location);

      return {
        ...summary,
        raw: entry.raw,
        score,
      };
    })
    .filter((item) => item.score > 0);

  const regionConstrainedCandidates = explicitRegionTokens.length
    ? candidates.filter((item) => itemMatchesExplicitRegion(item.raw, explicitRegionTokens))
    : candidates;

  const sorted = sortByScore(regionConstrainedCandidates);
  const grouped = { activity: [], scenic: [], hotel: [], product: [] };

  sorted.forEach((candidate) => {
    if (grouped[candidate.type].length >= MAX_CANDIDATE_PER_TYPE) return;
    grouped[candidate.type].push(candidate);
  });

  const selected = requestedTypes
    .flatMap((type) => grouped[type] || [])
    .concat(
      Object.keys(grouped)
        .filter((type) => !requestedTypes.includes(type))
        .flatMap((type) => grouped[type] || [])
    )
    .slice(0, 6);

  return {
    requestedTypes,
    intentTokens,
    regionLabel: explicitRegion || buildRegionLabel(location),
    selected,
  };
}

export async function buildPlatformGroundingContext(payload = {}) {
  const question = normalizeText(payload.question);
  if (!question) {
    return {
      question: "",
      regionLabel: DEFAULT_REGION_LABEL,
      candidates: [],
      prompt: "",
    };
  }

  const location = mergeSkillLocation(payload.location || {}, payload);
  const userProfile = payload.userProfile || {};
  const preferences = payload.preferences || {};
  const datasets = payload.platformDataset || (await loadPlatformDataset());
  const explicitRegion = readStateValue(payload, ["destination", "region", "place"]);
  const ranked = buildRankedCandidates({
    question,
    location,
    userProfile,
```

```
    datasets,
    explicitRegion,
  });

  const preferenceText = [
    preferences.distance ? `距离偏好：${preferences.distance}` : "",
    preferences.budget ? `预算偏好：${preferences.budget}` : "",
    preferences.detailLevel ? `回答详细度：${preferences.detailLevel}` : "",
  ]
    .filter(Boolean)
    .join("；");

  const profileText = normalizeArray(userProfile.dnaTags).join("、");
  const lines = ranked.selected.map((item, index) => formatCandidateLine(item, index));
  const dataBlock = lines.length ? lines.join("\n") : "平台当前没有筛到匹配候选。";

  const prompt = [
    "以下是平台当前可用的候选内容，请优先基于这些内容回答。",
    `用户当前问题：${question}`,
    `用户地区：${ranked.regionLabel}`,
    profileText ? `用户画像标签：${profileText}` : "",
    preferenceText ? `用户偏好：${preferenceText}` : "",
    ranked.intentTokens.length ? `问题意图关键词：${ranked.intentTokens.join("、")}` : "",
    "你不得编造平台中不存在的活动、景点、商品或酒店。",
    dataBlock,
  ]
    .filter(Boolean)
    .join("\n");

  return {
    question,
    regionLabel: ranked.regionLabel,
    candidates: ranked.selected,
    prompt,
  };
}

export function detectMainline(question = "", contextPayload = {}) {
  const currentTaskMainline = normalizeText(getCurrentTaskState(contextPayload)?.mainline);
  if (currentTaskMainline) {
    return currentTaskMainline;
  }

  const inferredMainline = inferMainlineFromHistory(contextPayload);
  if (inferredMainline) {
    return inferredMainline;
  }

  const skillMode = normalizeText(contextPayload?.skillContext?.mode);
  if (skillMode && MAINLINE_SKILL_MODE_MAP[skillMode]) {
    return MAINLINE_SKILL_MODE_MAP[skillMode];
  }

  const text = normalizeText(question);
  if (!text) return MAINLINE.GENERIC;
  if (includesAny(text, WEATHER_KEYWORDS)) return MAINLINE.WEATHER;
  if (isBuddyIntentText(text)) return MAINLINE.BUDDY;
  if (includesAny(text, GUIDE_KEYWORDS)) return MAINLINE.GUIDE;
  if (includesAny(text, FEEDBACK_KEYWORDS)) return MAINLINE.FEEDBACK;
  return MAINLINE.GENERIC;
}
```

```
function extractDestination(question = "", contextPayload = {}) {
  const fromCollected = readStateValue(contextPayload, ["destination", "region", "place"]);
  if (fromCollected) return fromCollected;

  const normalizedQuestion = normalizeText(question);
  const patterns = [
    /去([\u4e00-\u9fa5A-Za-z0-9]{2,12}?)(?:玩|逛|旅游|旅行|看|参加|的|，|。|\s|$)/u,
    /到([\u4e00-\u9fa5A-Za-z0-9]{2,12}?)(?:玩|逛|旅游|旅行|看|参加|的|，|。|\s|$)/u,
    /在([\u4e00-\u9fa5A-Za-z0-9]{2,12}?)(?:玩|逛|旅游|旅行)/u,
  ];

  for (const pattern of patterns) {
    const matched = normalizedQuestion.match(pattern);
    if (matched?.[1]) return matched[1];
  }

  if (
    /^[\u4e00-\u9fa5A-Za-z0-9]{2,12}$/u.test(normalizedQuestion) &&
    !extractTime(normalizedQuestion, contextPayload) &&
    !extractGuideBudgetValue(normalizedQuestion, contextPayload) &&
    !extractCount(normalizedQuestion, contextPayload, ["peopleCount"], /([一二三四五六七八九十两\d]+(?:人|位))/u)
  ) {
    return normalizedQuestion;
  }

  return "";
}

function extractGuideBudgetValue(question = "", contextPayload = {}) {
  const fromCollected = readStateValue(contextPayload, ["budget"]);
  if (fromCollected) return fromCollected;

  const normalizedQuestion = normalizeText(question);
  const amountWithUnit = normalizedQuestion.match(/(\d{2,5}\s*(?:元|块|w|万))/u);
  if (amountWithUnit?.[1]) return amountWithUnit[1];

  const embeddedBudget = normalizedQuestion.match(
    /(?:预算|人均|花费|费用|开销)(?:大概|大约|差不多|控制在)?\s*(\d{2,5})(?:\s*(?:元|块))?(?:左右|以内|以下|上下|上下浮动)?/u
  );
  if (embeddedBudget?.[1]) {
    return `${embeddedBudget[1]}元`;
  }

  const standaloneBudget = normalizedQuestion.match(
    /^(?:预算)?\s*(\d{2,5})(?:\s*(?:元|块))?(?:左右|以内|以下|上下|以左|以上)?$/u
  );
  if (standaloneBudget?.[1]) {
    return `${standaloneBudget[1]}元`;
  }

  const currentTaskState = getCurrentTaskState(contextPayload);
  if (normalizeText(currentTaskState?.lastAskedField) === "budget") {
    const plainNumber = normalizedQuestion.match(/^(\d{2,5})(?:左右|以内|以下|上下)?$/u);
    if (plainNumber?.[1]) {
      return `${plainNumber[1]}元`;
    }
  }

```

```
  return "";
}

function extractCount(question = "", contextPayload = {}, keys = [], pattern) {
  const fromCollected = readStateValue(contextPayload, keys);
  if (fromCollected) return fromCollected;
  const normalizedQuestion = normalizeText(question);
  const matched = normalizedQuestion.match(pattern);
  if (matched?.[1]) return matched[1];

  const extendedMatched = normalizedQuestion.match(/([一二三四五六七八九十两\d]+)\s*个?人/u);
  if (extendedMatched?.[1]) {
    return `${extendedMatched[1]}人`;
  }

  if (/^(我俩|我们)$/u.test(normalizedQuestion)) return "2人";
  if (/^(两个人|两人|2个人|2人|我们两个人|我们两人)$/u.test(normalizedQuestion)) return "2人";
  if (/^我和(?:男朋友|女朋友|对象|恋人|爱人|老公|老婆|朋友|同学|同事)/u.test(normalizedQuestion)) {
    return "2人";
  }

  return "";
}

function extractTime(question = "", contextPayload = {}) {
  const fromCollected = readStateValue(contextPayload, ["time", "travelTime", "date", "days"]);
  if (fromCollected) return fromCollected;

  const matched = normalizeText(question).match(
    /(今天|明天|后天|周末|五一|十一|端午|中秋|春节|暑假|寒假|下周|这周|周[一二三四五六日天]|\d{1,2}月\d{1,2}日|\d{1,2}号)/u
  );
  return matched?.[1] || "";
}

function extractCompanionPreference(question = "", contextPayload = {}) {
  const fromCollected = readStateValue(contextPayload, ["companionPreference", "groupPreference", "buddyPreference"]);
  if (fromCollected) return fromCollected;

  const normalizedQuestion = normalizeText(question);
  const directPatterns = [
    /(女生搭子|男生搭子|情侣搭子|亲子搭子|摄影搭子|饭搭子|自由行搭子)/u,
    /(同龄人|大学生|本地人|会开车|能拼车|随和一点|话少一点|会拍照|会摄影)/u,
  ];

  for (const pattern of directPatterns) {
    const matched = normalizedQuestion.match(pattern);
    if (matched?.[1]) return matched[1];
  }

  const normalizedGender =
    /(女生|小姐姐)/u.test(normalizedQuestion) ? "女生" : /(男生|小哥哥)/u.test(normalizedQuestion) ? "男生" : "";
  const normalizedTrait =
    /(会摄影|会拍照|摄影)/u.test(normalizedQuestion)
      ? "会摄影"
      : /(会开车|能开车)/u.test(normalizedQuestion)
        ? "会开车"
        : /能拼车/u.test(normalizedQuestion)
          ? "能拼车"
          : /同龄人/u.test(normalizedQuestion)
```

```
            ? "同龄人"
            : /大学生/u.test(normalizedQuestion)
              ? "大学生"
              : /本地人/u.test(normalizedQuestion)
                ? "本地人"
                : /随和/u.test(normalizedQuestion)
                  ? "随和一点"
                  : /话少/u.test(normalizedQuestion)
                    ? "话少一点"
                    : "";

  if (normalizedTrait && normalizedGender) {
    return `${normalizedTrait}的${normalizedGender}`;
  }

  if (normalizedTrait) return normalizedTrait;
  if (normalizedGender) return normalizedGender;
  return "";
}

function extractRelationship(question = "", contextPayload = {}) {
  const fromCollected = readStateValue(contextPayload, ["relationship", "groupType"]);
  if (fromCollected) return fromCollected;

  const normalizedQuestion = normalizeText(question);
  const matched = normalizedQuestion.match(
    /(情侣|夫妻|亲子|朋友|闺蜜|同学|同事|家人|一个人|独自|男朋友|女朋友|对象|恋人|爱人)/u
  );
  if (!matched?.[1]) return "";

  if (/(男朋友|女朋友|对象|恋人|爱人)/u.test(matched[1])) {
    return "情侣";
  }

  return matched[1];
}

function classifyFeedbackType(question = "") {
  const text = normalizeText(question);
  if (!text) return "普通反馈";
  if (/(建议|希望|能不能|可不可以|改进)/u.test(text)) return "平台建议";
  if (/(难用|卡|bug|闪退|进不去|加载慢|失败)/u.test(text)) return "普通反馈";
  if (/(烦|难受|委屈|伤心|emo|崩溃|累)/u.test(text)) return "情绪倾诉";
  return "普通反馈";
}

function normalizePeopleCountFromRelationship(relationship = "", currentPeopleCount = "")
  {
  if (normalizeText(currentPeopleCount)) {
    return normalizeText(currentPeopleCount);
  }

  const normalizedRelationship = normalizeText(relationship);
  if (normalizedRelationship === "情侣" || normalizedRelationship === "夫妻") {
    return "2人";
  }

  return "";
}

function buildBuddyWorkflow(question = "", contextPayload = {}) {
  const location = contextPayload?.location || {};
```

```
  const fields = {
    departure:
      readStateValue(contextPayload, ["departure", "origin"]) ||
      normalizeText(location.city || location.displayName),
    destination: extractDestination(question, contextPayload),
    time: extractTime(question, contextPayload),
    companionPreference: extractCompanionPreference(question, contextPayload),
  };

  const fieldDefs = [
    { key: "departure", label: "出发地", ask: "你这次准备从哪里出发？" },
    { key: "destination", label: "目的地", ask: "你这次最想去哪里？" },
    { key: "time", label: "时间", ask: "你大概什么时候出发？" },
    { key: "companionPreference", label: "同行偏好", ask: "你更想找什么样的同行搭子？" },
  ];

  return {
    mainline: MAINLINE.BUDDY,
    fields,
    missingField: fieldDefs.find((item) => !fields[item.key]) || null,
  };
}

function buildGuideWorkflow(question = "", contextPayload = {}) {
  const relationship = extractRelationship(question, contextPayload);
  const fields = {
    time: extractTime(question, contextPayload),
    peopleCount: extractCount(question, contextPayload, ["peopleCount"], /([一二三四五六七八九十两\d]+(?:人|位))/u),
    relationship,
    budget: extractGuideBudgetValue(question, contextPayload),
    destination: extractDestination(question, contextPayload),
  };
  fields.peopleCount = normalizePeopleCountFromRelationship(relationship, fields.peopleCount);

  const fieldDefs = [
    { key: "destination", label: "目的地", ask: "你这次最想去哪里？" },
    { key: "time", label: "时间", ask: "你打算什么时候去？" },
    { key: "peopleCount", label: "人数", ask: "这次大概几个人一起？" },
    { key: "relationship", label: "关系", ask: "这次同行的人大概是什么关系？" },
    { key: "budget", label: "预算", ask: "这次预算大概想控制在什么范围？" },
  ];

  return {
    mainline: MAINLINE.GUIDE,
    fields,
    missingField: fieldDefs.find((item) => !fields[item.key]) || null,
  };
}

function buildBuddyCandidateCard(currentUser = {}, candidate = {}, fields = {}) {
  const currentTags = new Set(normalizeArray(currentUser.dnaTags));
  const candidateTags = normalizeArray(candidate.dnaTags);
  const sharedTags = candidateTags.filter((tag) => currentTags.has(tag));
  const currentCity = removeRegionSuffix(currentUser.city || currentUser.province);
  const candidateCity = removeRegionSuffix(candidate.city || candidate.province);
  const destination = normalizeText(fields.destination);
  const companionPreference = normalizeText(fields.companionPreference);
  const candidateText = joinSearchText(candidate);

  let score = 40;
```

```
  if (currentCity && candidateCity && currentCity === candidateCity) score += 18;
  score += Math.min(sharedTags.length * 10, 30);
  if (companionPreference && candidateText.includes(normalizeTextLower(companionPreference))) score += 15;
  if (destination && candidateText.includes(normalizeTextLower(destination))) score += 10;
  if (candidate.profileCompleted) score += 5;
  if (candidate.dnaCompleted) score += 5;

  const reasons = [];
  if (currentCity && candidateCity && currentCity === candidateCity) reasons.push("同城出发更方便");
  if (sharedTags.length) reasons.push(`DNA 标签重合 ${Math.min(sharedTags.length, 3)} 项`);
  if (companionPreference && candidateText.includes(normalizeTextLower(companionPreference))) {
    reasons.push(`偏好贴近 “${companionPreference}” `);
  }
  if (!reasons.length) reasons.push("资料和偏好方向比较接近");

  return {
    id: normalizeText(candidate._id || candidate.id),
    type: "buddy",
    title: normalizeText(candidate.nickName) || "同行搭子",
    region:
      [candidate.province, candidate.city, candidate.district]
        .map(removeRegionSuffix)
        .filter(Boolean)
        .join(" / ") || "地区待补充",
    priceText: "同行匹配",
    summary: normalizeText(candidate.signature || candidate.bio || "") || "已完善基础资料，可进一步发起同行申请。",
    tags: uniqueList(sharedTags.concat(candidateTags)).slice(0, 4),
    matchReason: reasons.join("；"),
    score,
  };
}

async function buildBuddyCandidates(contextPayload = {}, cloudbaseUserId = "", workflow = {}) {
  const currentUser = await fetchUserByOpenid(cloudbaseUserId);
  if (!currentUser) {
    const users = await fetchCollection("users");
    console.warn("[agent-yuxiaohe] buildBuddyCandidates current user missing", {
      cloudbaseUserId: maskIdentifier(cloudbaseUserId),
      totalUsers: users.length,
      usersWithOpenid: users.filter((item) => normalizeText(item?.openid)).length,
      usersWith_Openid: users.filter((item) => normalizeText(item?._openid)).length,
      usersWithMatchingOpenid: users.filter(
        (item) => normalizeText(item?.openid) === normalizeText(cloudbaseUserId)
      ).length,
      usersWithMatching_Openid: users.filter(
        (item) => normalizeText(item?._openid) === normalizeText(cloudbaseUserId)
      ).length,
    });
    return [];
  }

  const users = await fetchCollection("users");
  const normalizedCloudbaseUserId = normalizeText(cloudbaseUserId);
  const withOpenid = users.filter((item) => normalizeText(item?.openid));
  const excludingSelf = withOpenid.filter(
    (item) => normalizeText(item?.openid) !== normalizedCloudbaseUserId
```

```
  );
  const profileCompletedUsers = excludingSelf.filter((item) => item?.profileCompleted);
  const dnaCompletedUsers = profileCompletedUsers.filter((item) => item?.dnaCompleted);

  console.log("[agent-yuxiaohe] buildBuddyCandidates stats", {
    cloudbaseUserId: maskIdentifier(normalizedCloudbaseUserId),
    currentUserFound: true,
    currentUserOpenid: maskIdentifier(currentUser?.openid || currentUser?._openid || "")
,
    currentUserProfileCompleted: Boolean(currentUser?.profileCompleted),
    currentUserDnaCompleted: Boolean(currentUser?.dnaCompleted),
    workflowFields: {
      departure: normalizeText(workflow?.fields?.departure),
      destination: normalizeText(workflow?.fields?.destination),
      time: normalizeText(workflow?.fields?.time),
      companionPreference: normalizeText(workflow?.fields?.companionPreference),
    },
    totalUsers: users.length,
    usersWithOpenid: withOpenid.length,
    usersWith_Openid: users.filter((item) => normalizeText(item?._openid)).length,
    excludingSelf: excludingSelf.length,
    profileCompleted: profileCompletedUsers.length,
    dnaCompleted: dnaCompletedUsers.length,
    finalCandidatePool: dnaCompletedUsers.length,
    sampleCandidateOpenids: dnaCompletedUsers
      .slice(0, 5)
      .map((item) => maskIdentifier(item?.openid || item?._openid || "")),
  });

  return dnaCompletedUsers
    .map((item) => buildBuddyCandidateCard(currentUser, item, workflow.fields || {}))
    .sort((left, right) => right.score - left.score)
    .slice(0, BUDDY_MAX_CANDIDATES);
}

export async function buildConversationWorkflow({
  question = "",
  contextPayload = {},
  cloudbaseUserId = "",
} = {}) {
  const mainline = detectMainline(question, contextPayload);
  const currentTaskState = getCurrentTaskState(contextPayload);

  if (mainline === MAINLINE.BUDDY) {
    const workflow = buildBuddyWorkflow(question, contextPayload);
    const buddyCandidates = workflow.missingField
      ? []
      : await buildBuddyCandidates(contextPayload, cloudbaseUserId, workflow);

    return {
      ...workflow,
      isReady: !workflow.missingField,
      buddyCandidates,
      lastAskedField: normalizeText(currentTaskState.lastAskedField),
    };
  }

  if (mainline === MAINLINE.GUIDE) {
    const workflow = buildGuideWorkflow(question, contextPayload);
    return {
      ...workflow,
      isReady: !workflow.missingField,
```

```
      lastAskedField: normalizeText(currentTaskState.lastAskedField),
    };
  }

  if (mainline === MAINLINE.FEEDBACK) {
    return {
      mainline,
      feedbackType:
        normalizeText(currentTaskState.feedbackType) ||
        classifyFeedbackType(question),
    };
  }

  return {
    mainline,
    lastAskedField: normalizeText(currentTaskState.lastAskedField),
  };
}

export async function buildUnifiedTaskState({
  question = "",
  contextPayload = {},
  cloudbaseUserId = "",
} = {}) {
  const workflow = await buildConversationWorkflow({
    question,
    contextPayload,
    cloudbaseUserId,
  });
  const currentTaskState = getCurrentTaskState(contextPayload);

  return {
    mainline: normalizeText(workflow?.mainline || currentTaskState?.mainline),
    collected: workflow?.fields || currentTaskState?.collected || {},
    missingField: workflow?.missingField
      ? {
          key: normalizeText(workflow.missingField.key),
          label: normalizeText(workflow.missingField.label),
          ask: normalizeText(workflow.missingField.ask),
        }
      : null,
    lastAskedField: normalizeText(
      workflow?.missingField?.key || workflow?.lastAskedField || currentTaskState?.lastAskedField
    ),
    feedbackType: normalizeText(workflow?.feedbackType || currentTaskState?.feedbackType),
    isReady: Boolean(workflow?.isReady),
  };
}

export async function buildBuddyMatchResult({
  departure = "",
  destination = "",
  time = "",
  companionPreference = "",
  cloudbaseUserId = "",
  contextPayload = {},
} = {}) {
  const mergedContextPayload = {
    ...(contextPayload || {}),
    currentTaskState: {
```

```
      ...(contextPayload?.currentTaskState || {}),
      mainline: MAINLINE.BUDDY,
      collected: {
        ...(contextPayload?.currentTaskState?.collected || {}),
        departure: normalizeText(departure),
        destination: normalizeText(destination),
        time: normalizeText(time),
        companionPreference: normalizeText(companionPreference),
      },
    },
  };

  const workflow = await buildConversationWorkflow({
    question: "",
    contextPayload: mergedContextPayload,
    cloudbaseUserId,
  });

  return {
    success: true,
    mainline: workflow?.mainline || MAINLINE.BUDDY,
    isReady: !!workflow?.isReady,
    missingField: workflow?.missingField
      ? {
          key: normalizeText(workflow.missingField.key),
          label: normalizeText(workflow.missingField.label),
          ask: normalizeText(workflow.missingField.ask),
        }
      : null,
    collected: workflow?.fields || {},
    candidates: normalizeBuddyCandidates(workflow?.buddyCandidates),
  };
}

export function buildGuideRecallQuestion(fields = {}) {
  const segments = [
    normalizeText(fields.destination),
    normalizeText(fields.time),
    normalizeText(fields.peopleCount),
    normalizeText(fields.relationship),
    normalizeText(fields.budget),
    "活动",
    "景点",
    "住宿",
    "路线",
  ].filter(Boolean);

  return segments.join(" ");
}

export function buildGuideGroundingPayload({
  question = "",
  contextPayload = {},
  guideFields = {},
} = {}) {
  const normalizedFields =
    guideFields && typeof guideFields === "object" ? guideFields : {};

  return {
    ...(contextPayload || {}),
    question: buildGuideRecallQuestion(normalizedFields) || normalizeText(question),
    location: contextPayload?.location || {},
```

```
    userProfile: contextPayload?.userProfile || {},
    preferences: contextPayload?.preferences || {},
    skillContext: {
      ...(contextPayload?.skillContext || {}),
      mode: MAINLINE.GUIDE,
      title: normalizeText(contextPayload?.skillContext?.title) || "攻略定制",
      collected: {
        ...(contextPayload?.skillContext?.collected || {}),
        ...normalizedFields,
      },
    },
    currentTaskState: {
      ...(contextPayload?.currentTaskState || {}),
      mainline: MAINLINE.GUIDE,
      collected: {
        ...(contextPayload?.currentTaskState?.collected || {}),
        ...normalizedFields,
      },
    },
  };
}

function normalizeGuideCandidates(candidates = []) {
  return (Array.isArray(candidates) ? candidates : [])
    .filter((item) => normalizeText(item.type) !== "product")
    .slice(0, GUIDE_MAX_CANDIDATES)
    .map((item) => ({
      id: normalizeText(item.id),
      type: normalizeText(item.type),
      title: normalizeText(item.title),
      region: normalizeText(item.region),
      priceText: normalizeText(item.priceText),
      summary: normalizeText(item.summary),
      tags: normalizeArray(item.tags),
    }));
}

function normalizeBuddyCandidates(candidates = []) {
  return (Array.isArray(candidates) ? candidates : []).map((item) => ({
    id: normalizeText(item.id),
    nickname: normalizeText(item.title),
    region: normalizeText(item.region),
    summary: normalizeText(item.summary),
    tags: normalizeArray(item.tags),
    matchReason: normalizeText(item.matchReason),
  }));
}

export async function buildGuideCustomizationResult({
  time = "",
  peopleCount = "",
  relationship = "",
  budget = "",
  destination = "",
  contextPayload = {},
} = {}) {
  const mergedContextPayload = {
    ...(contextPayload || {}),
    currentTaskState: {
      ...(contextPayload?.currentTaskState || {}),
      mainline: MAINLINE.GUIDE,
      collected: {
```

```
        ...(contextPayload?.currentTaskState?.collected || {}),
        time: normalizeText(time),
        peopleCount: normalizeText(peopleCount),
        relationship: normalizeText(relationship),
        budget: normalizeText(budget),
        destination: normalizeText(destination),
      },
    },
  };

  const workflow = await buildConversationWorkflow({
    question: "",
    contextPayload: mergedContextPayload,
    cloudbaseUserId: "",
  });

  if (!workflow?.isReady) {
    return {
      success: true,
      mainline: workflow?.mainline || MAINLINE.GUIDE,
      isReady: false,
      missingField: workflow?.missingField
        ? {
            key: normalizeText(workflow.missingField.key),
            label: normalizeText(workflow.missingField.label),
            ask: normalizeText(workflow.missingField.ask),
          }
        : null,
      collected: workflow?.fields || {},
      candidates: [],
    };
  }

  const groundingContext = await buildPlatformGroundingContext(
    buildGuideGroundingPayload({
      question: buildGuideRecallQuestion(workflow.fields || {}),
      contextPayload: mergedContextPayload,
      guideFields: workflow.fields || {},
    })
  );

  return {
    success: true,
    mainline: workflow?.mainline || MAINLINE.GUIDE,
    isReady: true,
    missingField: null,
    collected: workflow?.fields || {},
    candidates: normalizeGuideCandidates(groundingContext?.candidates || []),
  };
}

export async function buildConversationStateResult({
  question = "",
  contextPayload = {},
  cloudbaseUserId = "",
} = {}) {
  const workflow = await buildConversationWorkflow({
    question,
    contextPayload,
    cloudbaseUserId,
  });
  const unifiedTaskState = await buildUnifiedTaskState({
```

```
    question,
    contextPayload,
    cloudbaseUserId,
  });

  let candidates = [];

  if (
    normalizeText(unifiedTaskState?.mainline) === MAINLINE.GUIDE &&
    Boolean(unifiedTaskState?.isReady)
  ) {
    const guideFields = workflow?.fields || unifiedTaskState?.collected || {};
    const groundingContext = await buildPlatformGroundingContext(
      buildGuideGroundingPayload({
        question,
        contextPayload,
        guideFields,
      })
    );

    candidates = normalizeGuideCandidates(groundingContext?.candidates || []);
  }

  if (
    normalizeText(unifiedTaskState?.mainline) === MAINLINE.BUDDY &&
    Boolean(unifiedTaskState?.isReady)
  ) {
    candidates = normalizeBuddyCandidates(workflow?.buddyCandidates);
  }

  return {
    success: true,
    mainline: normalizeText(unifiedTaskState?.mainline),
    collected: unifiedTaskState?.collected || {},
    missingField: unifiedTaskState?.missingField || null,
    lastAskedField: normalizeText(unifiedTaskState?.lastAskedField),
    feedbackType: normalizeText(unifiedTaskState?.feedbackType),
    isReady: Boolean(unifiedTaskState?.isReady),
    candidates,
  };
}

export function buildAgentUserPrompt({
  question = "",
  groundingContext = {},
  workflowContext = {},
  contextPayload = {},
} = {}) {
  const normalizedQuestion = normalizeText(question);
  const regionLabel = normalizeText(groundingContext?.regionLabel) || DEFAULT_REGION_LAB
EL;
  const profileTags = normalizeArray(contextPayload?.userProfile?.dnaTags).join("、");
  const preferencePairs = Object.entries(contextPayload?.preferences || {})
    .map(([key, value]) => [normalizeText(key), normalizeText(value)])
    .filter(([, value]) => value)
    .map(([key, value]) => `${key}: ${value}`);
  const skillMode = normalizeText(contextPayload?.skillContext?.mode);
  const skillTitle = normalizeText(contextPayload?.skillContext?.title);
  const skillPairs = Object.entries(getMergedCollected(contextPayload))
    .map(([key, value]) => [normalizeText(key), normalizeText(value)])
    .filter(([, value]) => value)
    .map(([key, value]) => `${key}: ${value}`);
```

```
  const mainline = workflowContext?.mainline || detectMainline(question, contextPayload);
  const shouldAttachPlatformGrounding =
    mainline === MAINLINE.GUIDE || mainline === MAINLINE.GENERIC;

  const buddyFields = workflowContext?.fields || {};
  const guideFields = workflowContext?.fields || {};
  const buddyCandidates = Array.isArray(workflowContext?.buddyCandidates)
    ? workflowContext.buddyCandidates
    : [];

  const mainlineInstruction =
    mainline === MAINLINE.BUDDY
      ? [
          "当前主线：找搭子。",
          "第一轮只问 1 个最必要的问题。",
          "目标字段只有：出发地、目的地、时间、同行偏好。",
          `当前字段：出发地=${buddyFields.departure || "未提供"}；目的地=${buddyFields.destination || "未提供"}；时间=${buddyFields.time || "未提供"}；同行偏好=${buddyFields.companionPreference || "未提供"}。`,
          workflowContext?.missingField
            ? `信息还不足，当前只追问这个问题：${workflowContext.missingField.ask}`
            : buddyCandidates.length
              ? `信息已足够，请仅基于这些搭子候选回答，并说明匹配理由：\n${buddyCandidates.map((item, index) => `${index + 1}. ${item.title} | ${item.region} | ${item.matchReason}`).join("\n")}`
              : "信息已足够，但当前没有合适候选。请明确说明“当前没有合适候选”，并建议补充时间、出发地或同行偏好。",
          "禁止跳成商品、景点、特产推荐。",
          "禁止因为“美食”等兴趣词误判成商品推荐。",
        ].join("\n")
      : mainline === MAINLINE.GUIDE
        ? [
            "当前主线：攻略定制。",
            "先收集：时间、人数、关系、预算、目的地。",
            `当前字段：时间=${guideFields.time || "未提供"}；人数=${guideFields.peopleCount || "未提供"}；关系=${guideFields.relationship || "未提供"}；预算=${guideFields.budget || "未提供"}；目的地=${guideFields.destination || "未提供"}。`,
            workflowContext?.missingField
              ? `当前信息不足，只问这一个关键问题：${workflowContext.missingField.ask}`
              : "信息足够后再生成路线和建议，并且必须解释“为什么推荐这些”。",
            "不得脑补平台中不存在的内容。",
          ].join("\n")
        : mainline === MAINLINE.FEEDBACK
          ? [
              "当前主线：树洞反馈。",
              `当前反馈分类：${workflowContext?.feedbackType || "普通反馈"}。`,
              "以接住情绪、接收意见为主。",
              "不导购、不推荐、不跳业务主线，只在必要时温和追问。",
            ].join("\n")
          : mainline === MAINLINE.WEATHER
            ? "当前主线：天气/位置。只负责天气、位置、穿衣和出行提醒，不扩题。"
            : [
                "当前主线：普通聊天防串场。",
                "不是天气意图不要查天气。",
                "不是商品意图不要推商品。",
                "不是搭子意图不要匹配搭子。",
                "不确定时先确认，不要自动跳任务。",
              ].join("\n");

  const sections = [
    "你是智能体“裕小禾”，对话中自称“小禾”。",
    "“问小禾”只是小程序里的功能入口名，不是你的名字。",
    "你必须优先使用平台候选和上下文，不得编造平台中不存在的内容。",
```

```
    "如果缺数据，要明确说明缺什么。",
    skillMode ? `当前对话来自技能流程：${skillTitle || skillMode}` : "",
    skillPairs.length ? `已收集任务字段：${skillPairs.join("; ")}` : "",
    `当前执行主线：${mainline}`,
    mainlineInstruction,
    `用户当前问题：${normalizedQuestion}`,
    `用户当前地区：${regionLabel}`,
    profileTags ? `用户画像标签：${profileTags}` : "",
    preferencePairs.length ? `用户偏好：${preferencePairs.join("; ")}` : "",
    shouldAttachPlatformGrounding ? groundingContext?.prompt || "" : "",
    mainline === MAINLINE.GUIDE && workflowContext?.isReady
      ? "攻略主线在信息齐全时，正文只允许输出简短承接、轻量解释和温馨提示，不允许用 1. 2. 3. 枚举推荐项，不允许重复复述具体候选项，由前端结构化卡片展示。"
      : "",
    "请直接开始回答用户。",
  ];

  return sections.filter(Boolean).join("\n\n");
}

export class DetectCloudbaseUserMiddleware {
  constructor(request) {
    this.request = request;
  }

  async beforeAgent({ input }) {
    const authorization =
      this.request?.headers?.authorization ||
      this.request?.headers?.Authorization ||
      this.request?.headers?.get?.("authorization") ||
      this.request?.headers?.get?.("Authorization") ||
      "";

    const token = authorization.startsWith("Bearer ") ? authorization.slice(7) : "";
    if (!token) return;

    try {
      const payload = jwtDecode(token);
      input.forwardedProps = {
        ...(input.forwardedProps || {}),
        cloudbaseUserId: payload?.uid || payload?.sub || payload?.openid || payload?.user_id || "",
      };
    } catch (error) {
      console.warn("[DetectCloudbaseUserMiddleware] jwt decode failed", error?.message || error);
    }
  }
}
```

??: cloudfunctions/agent-yuxiaohe-1grmumm967563411/package.json

```
{
  "name": "agent-yuxiaohe",
  "version": "1.0.0",
  "description": "CloudBase Agent service for 裕小禾",
  "main": "src/index.js",
  "type": "module",
  "scripts": {
    "start": "node src/index.js"
  },
  "dependencies": {
    "@cloudbase/agent-adapter-adp": "^1.0.1-alpha.30",
    "@cloudbase/agent-adapter-wx": "^1.0.1-alpha.30",
    "@cloudbase/agent-server": "^0.0.20",
    "@cloudbase/node-sdk": "^3.18.1",
    "@dotenvx/dotenvx": "^1.51.1",
    "express": "^5.2.1",
    "jwt-decode": "^4.0.0",
    "rxjs": "7.8.1"
  },
  "overrides": {
    "rxjs": "7.8.1"
  }
}
```